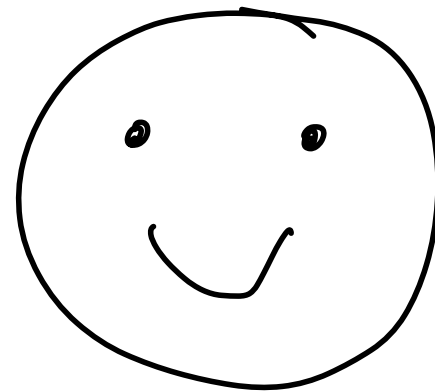


Dynamic Light Spanners in Doubling Metrics



Sujoy Bhore
IIT Bombay



Jonathan Conroy
Dartmouth College

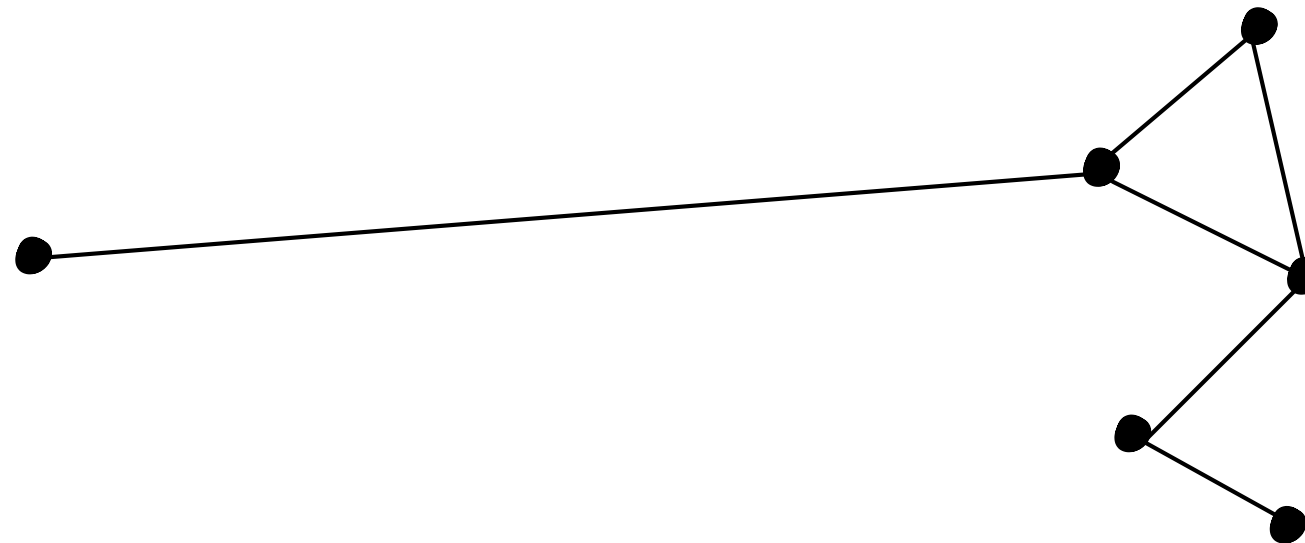


Arnold Filtser
Bar-Ilan University

Spanners



Spanners

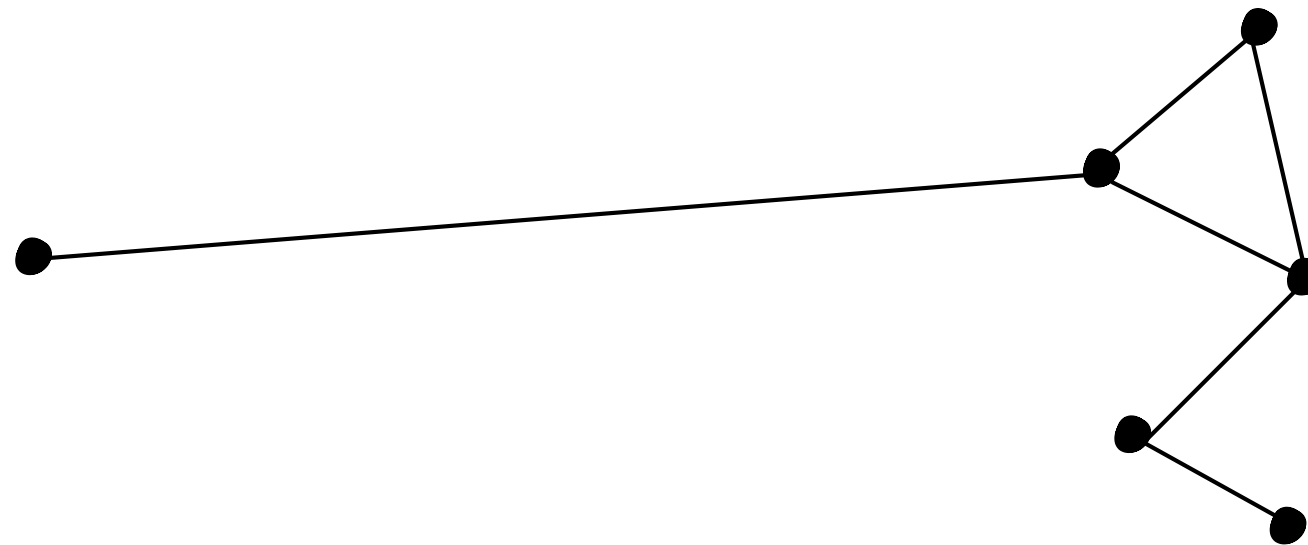


Spanners

$(1 + \varepsilon)$ -*spanner* is geometric graph G on P with

$$d_G(u, v) \leq (1 + \varepsilon) \|u - v\|$$

$$\forall u, v \in P$$



Spanners

$(1 + \varepsilon)$ -*spanner* is geometric graph G on P with

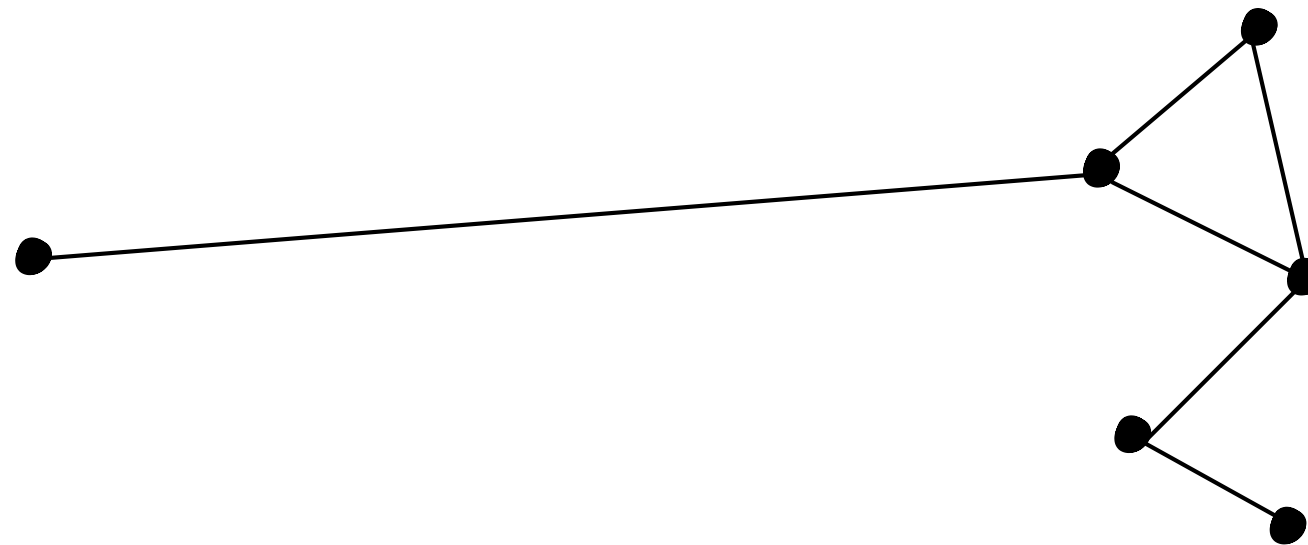
$$d_G(u, v) \leq (1 + \varepsilon) \|u - v\|$$

$$\forall u, v \in P$$

Sparsity

In $\mathbb{R}^d$: $O_d(1)$

[Ruppert & Seidel 1997]



Lightness

In $\mathbb{R}^d$: $O_d(1)$

[Das, Narasimhan, Salowe 1995]

Spanners

$(1 + \varepsilon)$ -*spanner* is geometric graph G on P with

$$d_G(u, v) \leq (1 + \varepsilon) \|u - v\|$$

$$\forall u, v \in P$$

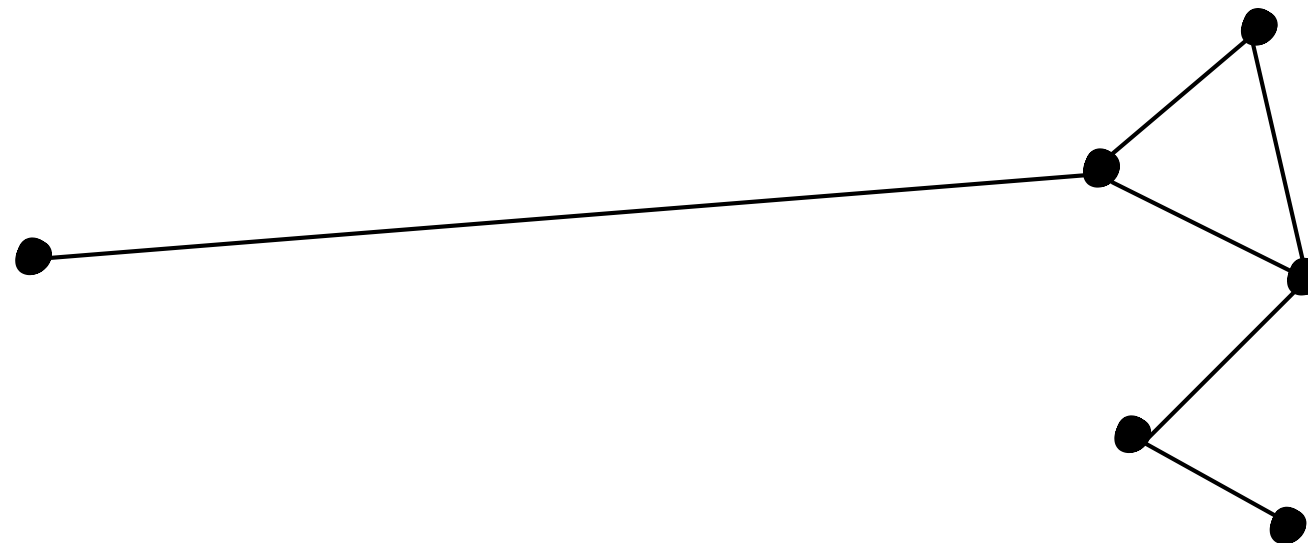
Sparsity

In $\mathbb{R}^d$: $O_d(1)$

[Ruppert & Seidel 1997]

Doubling dim d : $O_d(1)$

[Gao, Guibas, Nguyen 2005]



Lightness

In $\mathbb{R}^d$: $O_d(1)$

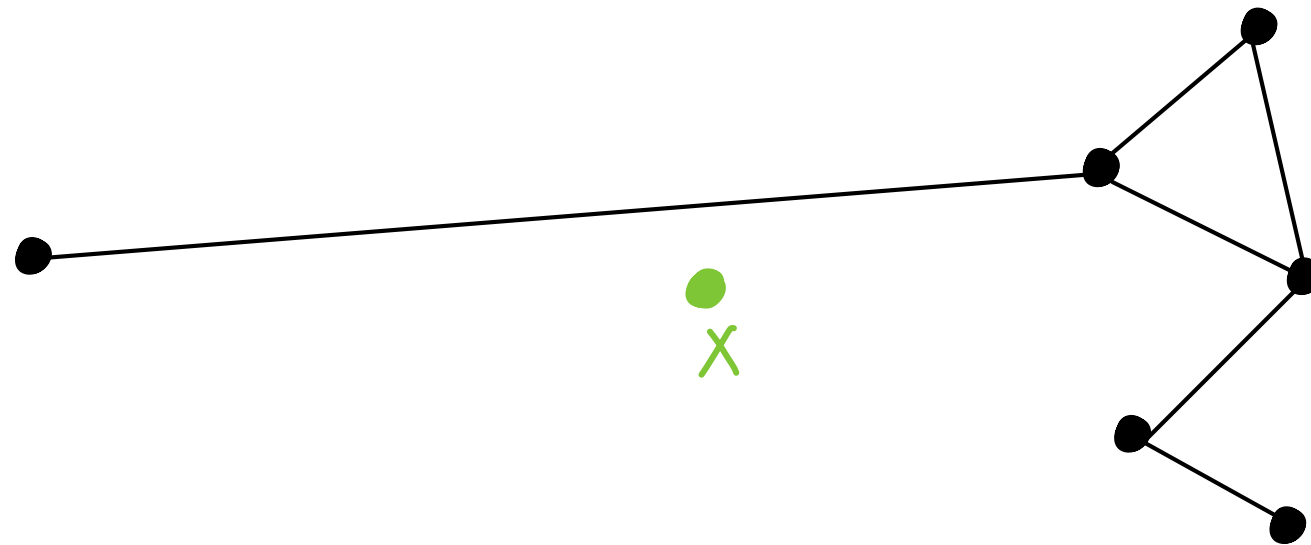
[Das, Narasimhan, Salowe 1995]

Doubling dim d : $O_d(1)$

[Cole and Gottlieb 2015]

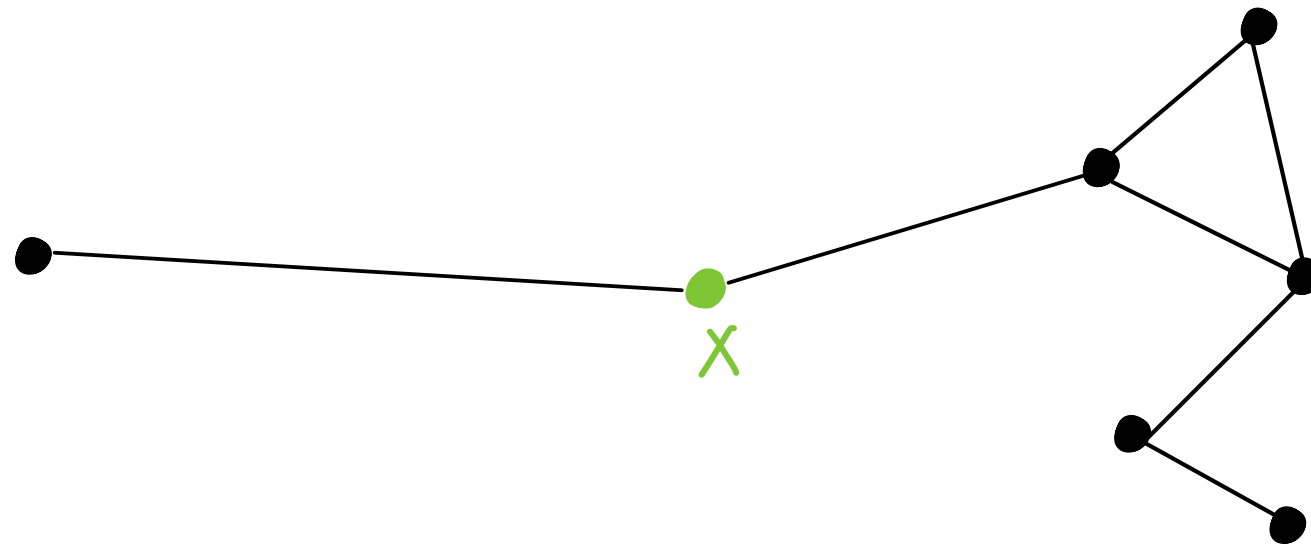
Dynamic Spanners

Points can be *inserted* or *deleted* in P



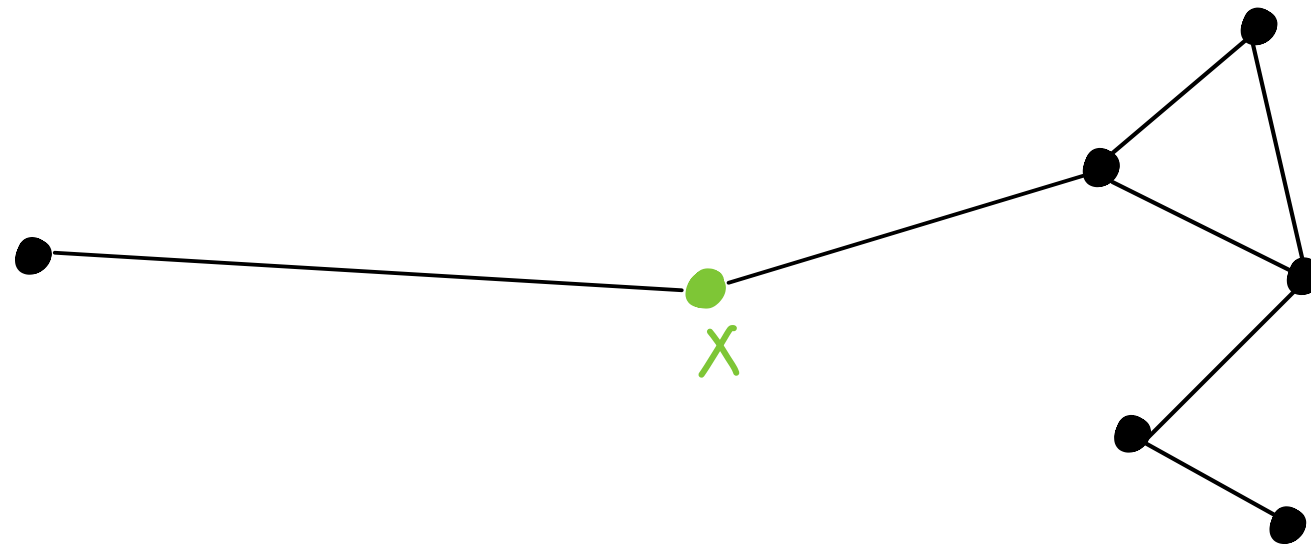
Dynamic Spanners

Points can be *inserted* or *deleted* in P



Dynamic Spanners

Points can be *inserted* or *deleted* in P

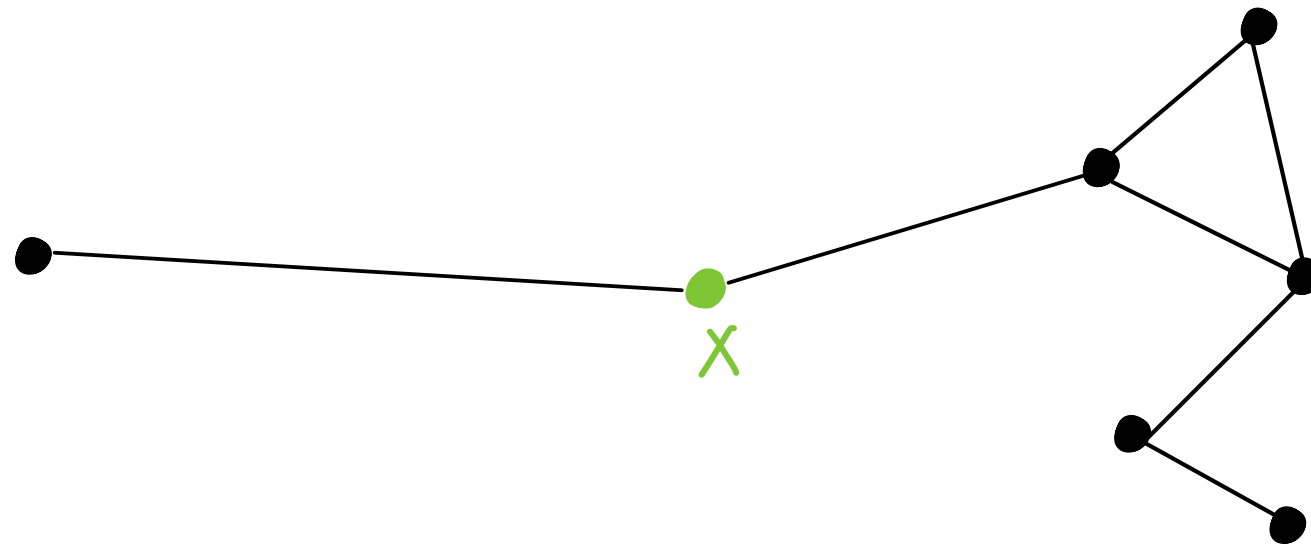


$O_d(1)$ sparsity
in $O_d(\log n)$ update time

$O_d(1)$ lightness
with $O(\log \Phi)$ recourse

Dynamic Spanners

Points can be *inserted* or *deleted* in P



$O_d(l)$ sparsity
in $O_d(\log n)$ update time

[Gao, Guibas, Nguyen 2005]

$O_d(l)$ lightness
with $O(\log \Phi)$ recourse
Spread of $\vec{P}$ $\downarrow$ # changing edges

[Eppstein and Khodabandeh 2024]

Our Result

Let P be a set of points in $\mathbb{R}^d$ undergoing insertions and deletions, with spread Φ . We maintain a $(1 + \varepsilon)$ -spanner with lightness $O_d(1)$ with update time $(\log \Phi)^{O(d)}$.

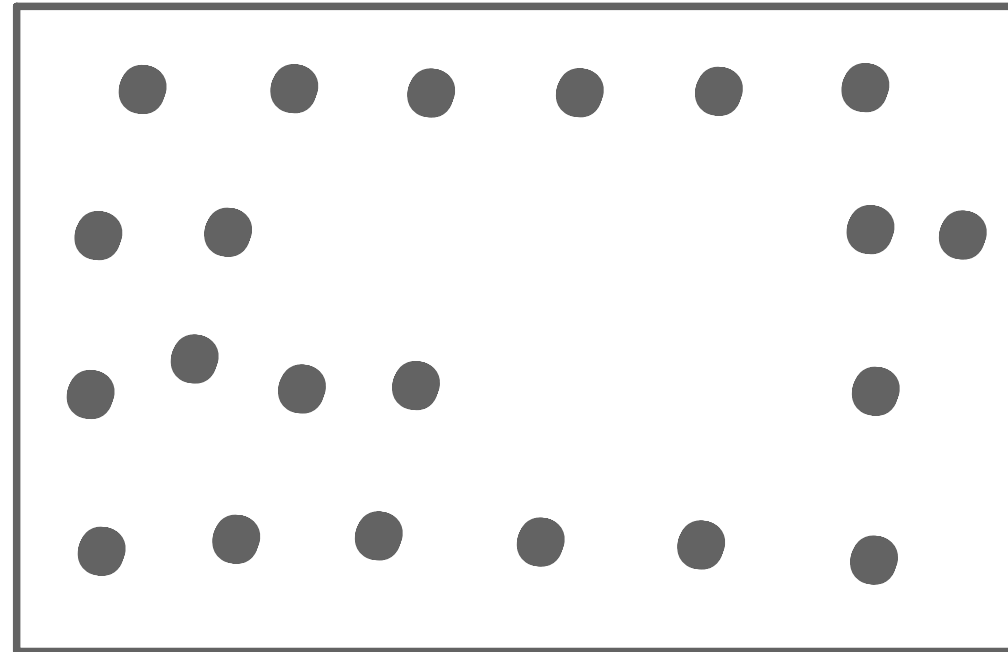
Outline

1. Background on Spanners

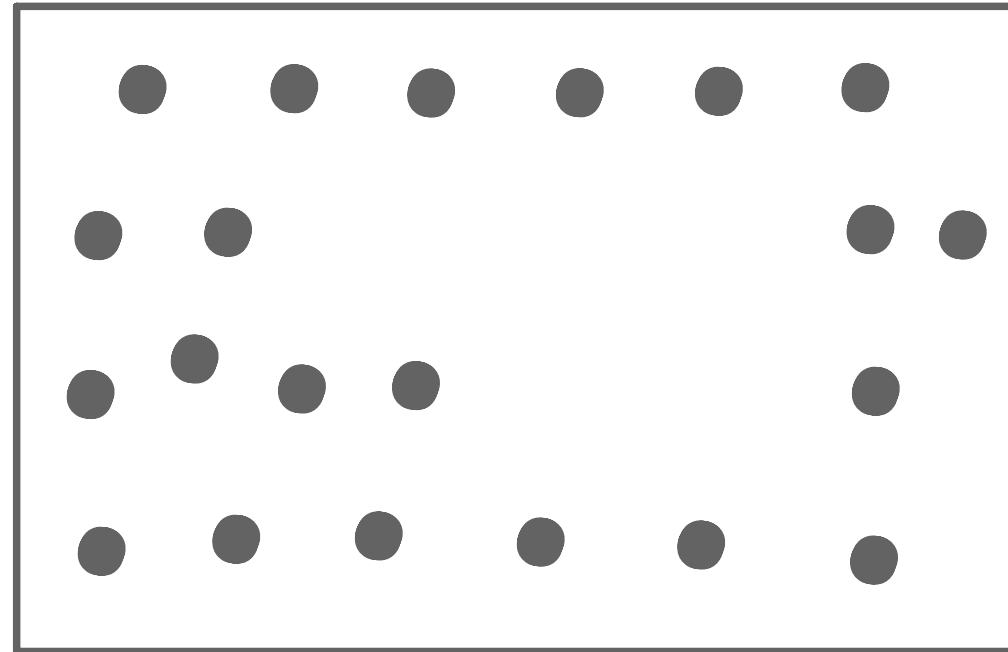
2. Spanner with $O(\log \Phi)$ Recourse

3. Updates in $\text{poly log } \Phi$ Time

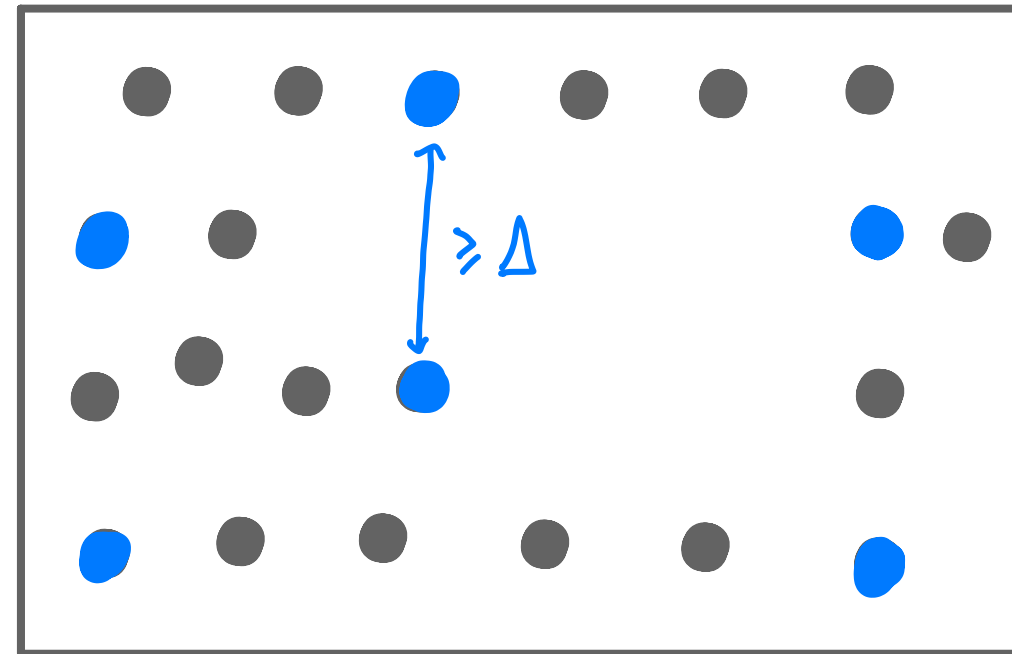
Background: Net-Tree Spanner _[GGN05]



Background: Net-Tree Spanner [GGN05]



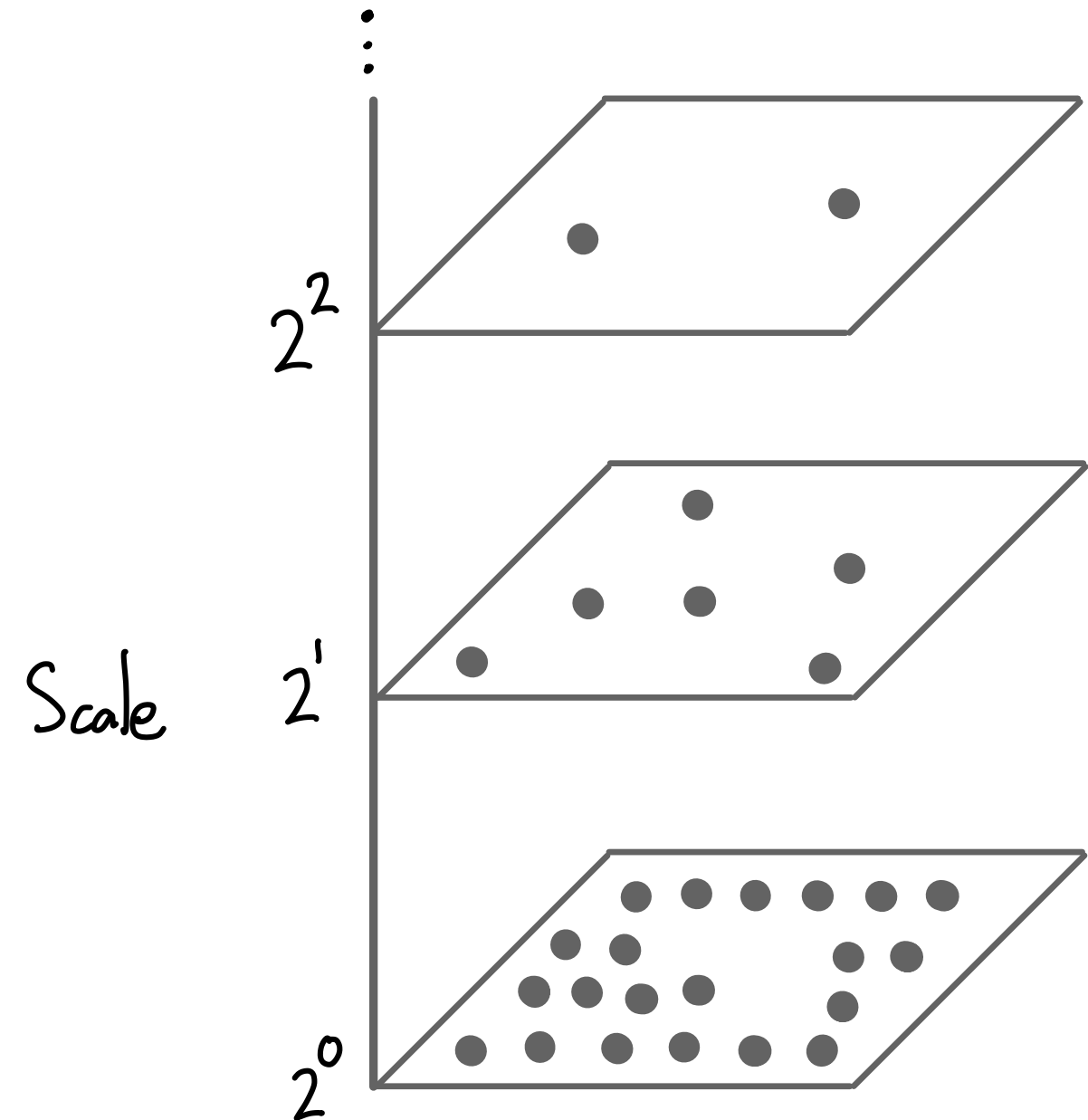
Background: Net-Tree Spanner _[GGN05]



Δ -net of P

Background: Net-Tree Spanner

Store hierarchy of nets N_i

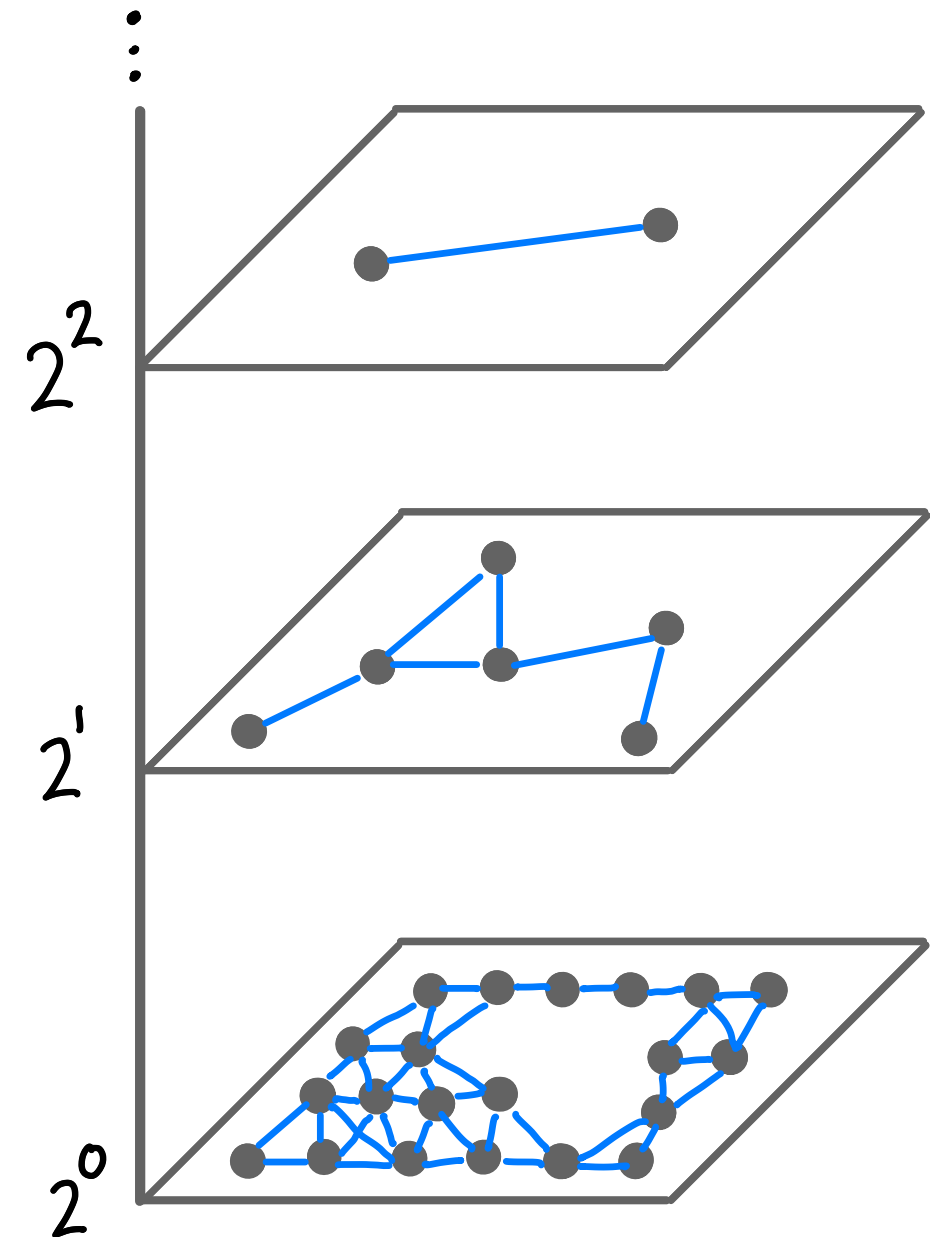


Background: Net-Tree Spanner

Store hierarchy of nets N_i

For each N_i , add edge
 (x, x') if $\|x - x'\| \leq O(2^i)$

Scale

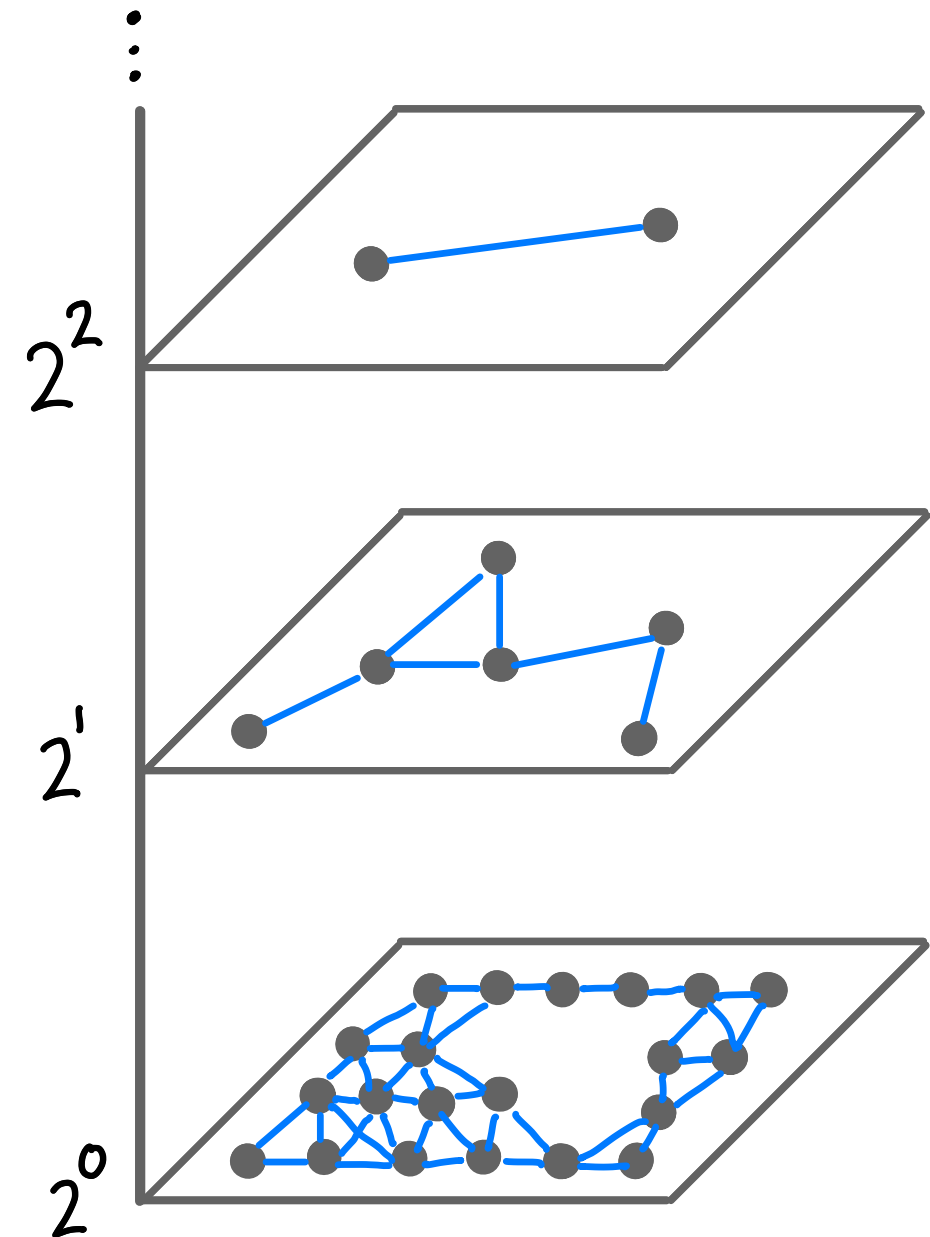


Background: Net-Tree Spanner

Store hierarchy of nets N_i

For each N_i , add edge
 (x, x') if $\|x - x'\| \leq O(2^i)$

Scale

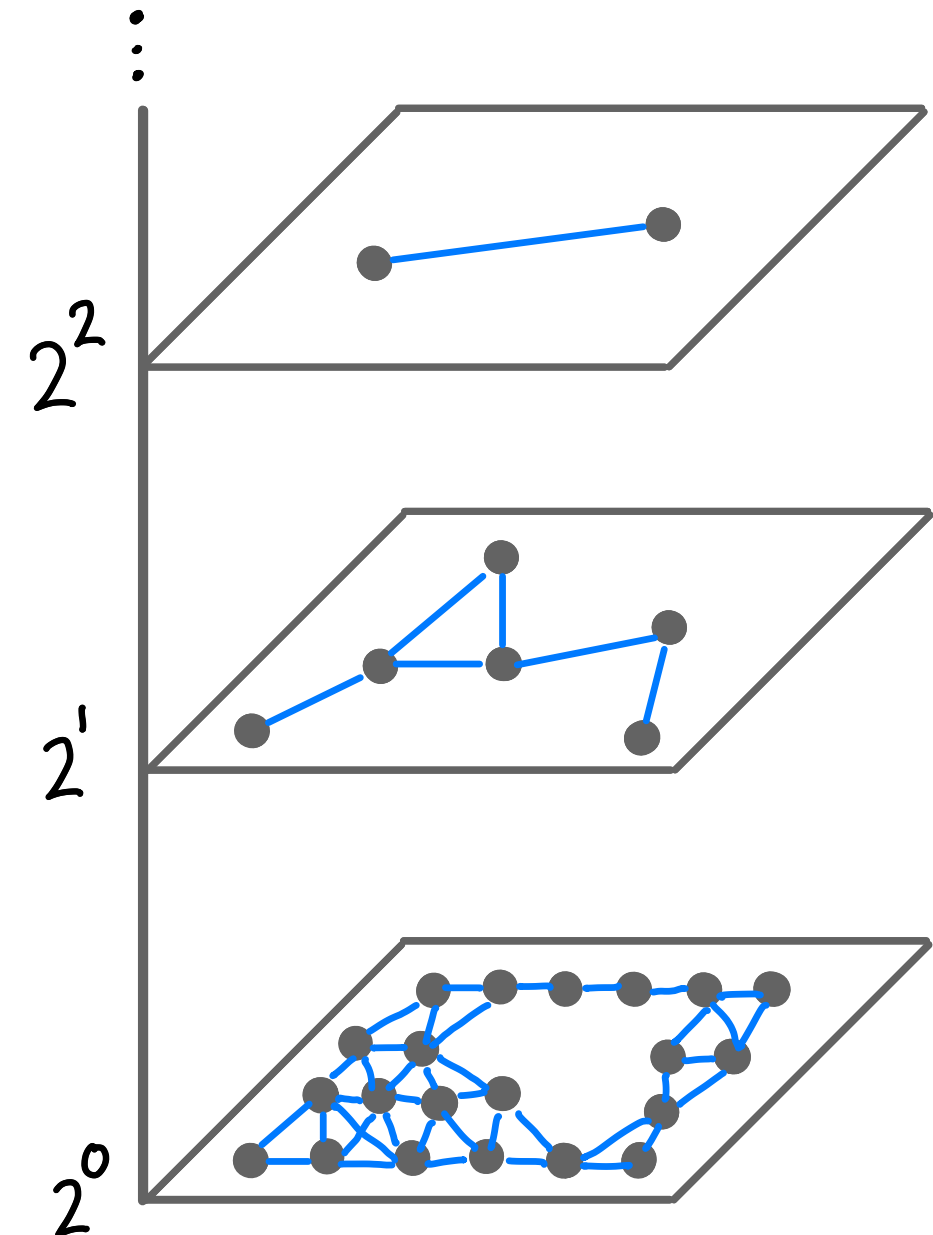


Background: Net-Tree Spanner

Store hierarchy of nets N_i

For each N_i , add edge
 (x, x') if $\|x - x'\| \leq O(2^i)$

Scale

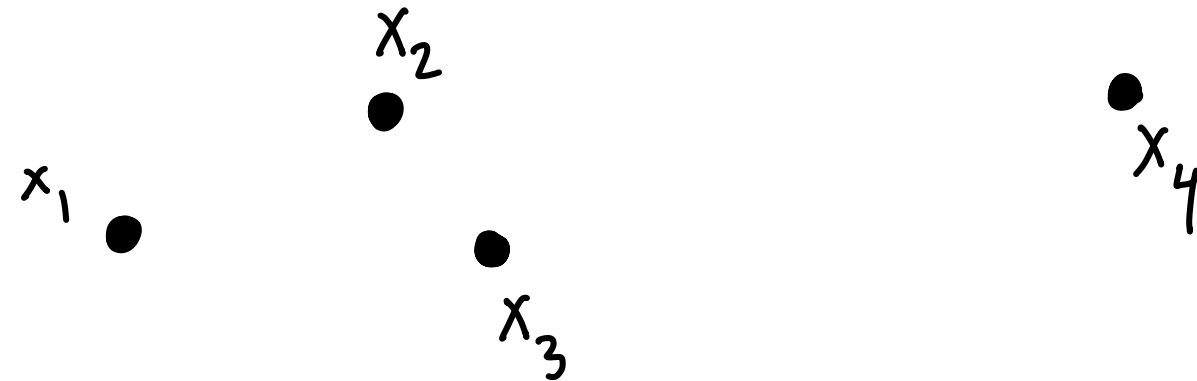


Net-tree spanner is $O_d(1)$ -sparse. Dynamic updates in $O(\log \Phi)$ time!

Background: Greedy Spanner

Greedy spanner

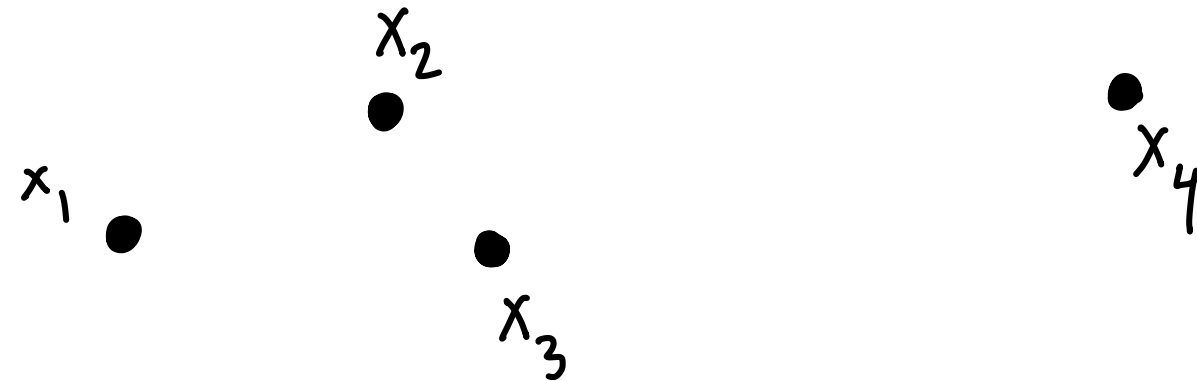
- Initialize $H \leftarrow (P, \emptyset)$.
- Examine all pairs $(u, v) \in P \times P$ (in increasing distance order): if $d_H(u, v) \leq (1 + \varepsilon)||u - v||$, add edge (u, v) to H



Background: Greedy Spanner

Greedy spanner

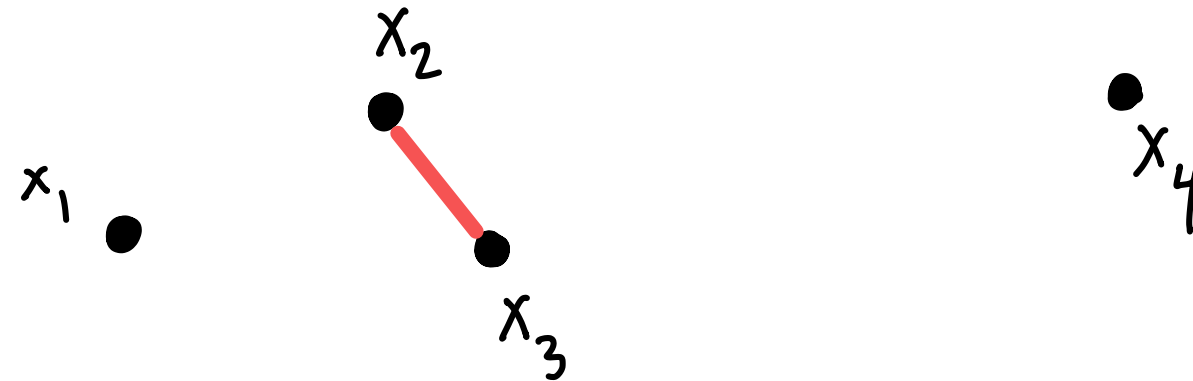
- Initialize $H \leftarrow (P, \emptyset)$.
- Examine all pairs $(u, v) \in P \times P$ (in increasing distance order): if $d_H(u, v) \leq (1 + \varepsilon)||u - v||$, add edge (u, v) to H



Background: Greedy Spanner

Greedy spanner

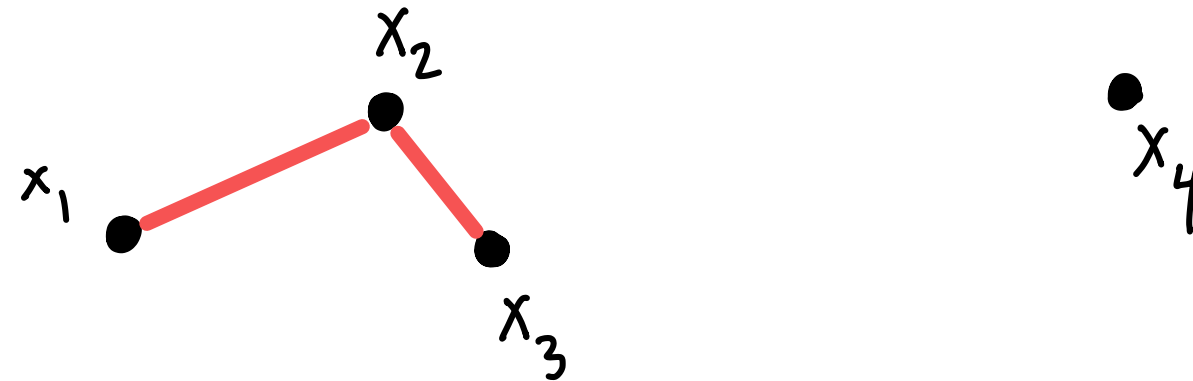
- Initialize $H \leftarrow (P, \emptyset)$.
- Examine all pairs $(u, v) \in P \times P$ (in increasing distance order): if $d_H(u, v) \leq (1 + \varepsilon)||u - v||$, add edge (u, v) to H



Background: Greedy Spanner

Greedy spanner

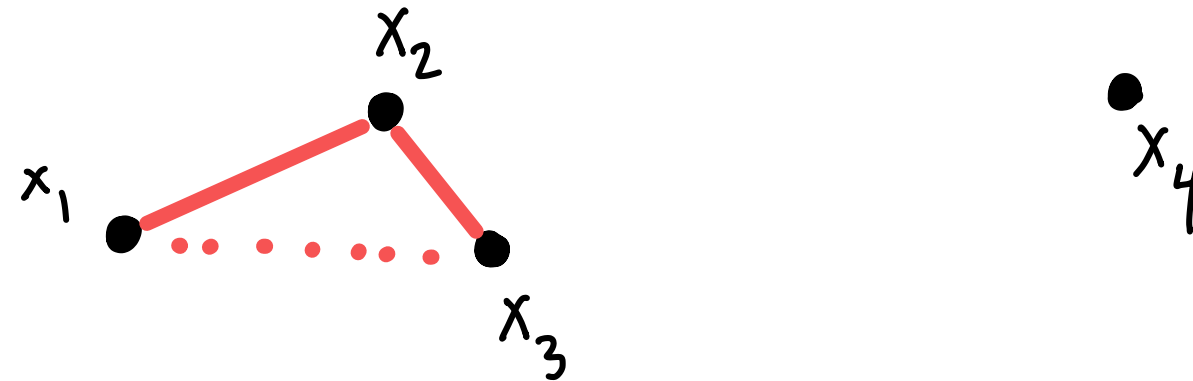
- Initialize $H \leftarrow (P, \emptyset)$.
- Examine all pairs $(u, v) \in P \times P$ (in increasing distance order): if $d_H(u, v) \leq (1 + \varepsilon)||u - v||$, add edge (u, v) to H



Background: Greedy Spanner

Greedy spanner

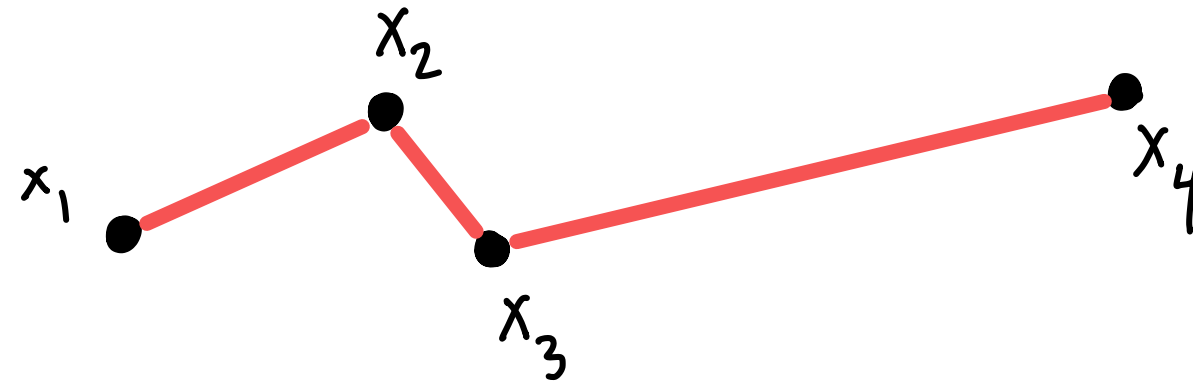
- Initialize $H \leftarrow (P, \emptyset)$.
- Examine all pairs $(u, v) \in P \times P$ (in increasing distance order): if $d_H(u, v) \leq (1 + \varepsilon)||u - v||$, add edge (u, v) to H



Background: Greedy Spanner

Greedy spanner

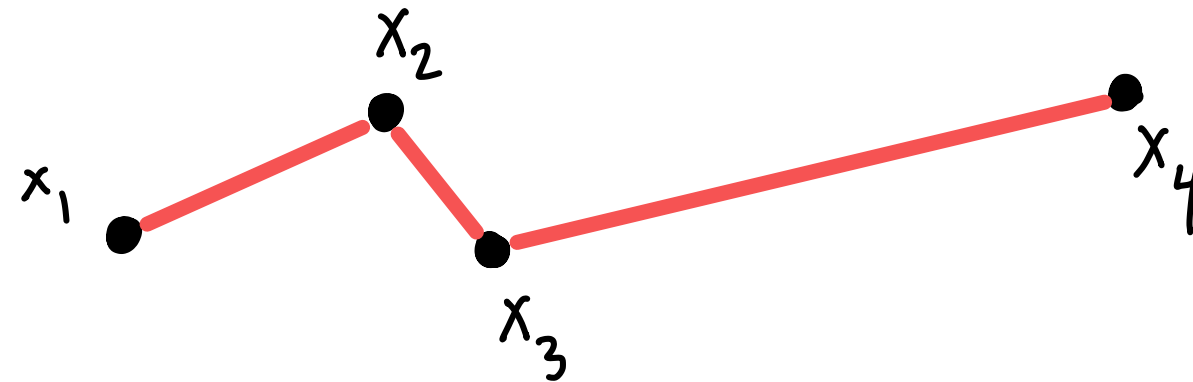
- Initialize $H \leftarrow (P, \emptyset)$.
- Examine all pairs $(u, v) \in P \times P$ (in increasing distance order): if $d_H(u, v) \leq (1 + \varepsilon)||u - v||$, add edge (u, v) to H



Background: Greedy Spanner

Greedy spanner

- Initialize $H \leftarrow (P, \emptyset)$.
- Examine all pairs $(u, v) \in P \times P$ (in increasing distance order): if $d_H(u, v) \leq (1 + \varepsilon)||u - v||$, add edge (u, v) to H



Greedy spanner has lightness $O_d(1)$. [Got15, FS20]

Background: Spanner of [EK24]

Result of [EK24]: One can maintain a $(1 + \varepsilon)$ spanner of P with lightness $O_d(1)$ and amortized recourse $O_d(\log \Phi)$.

Rough idea: Store a net-tree spanner H_{net} . Maintain greedy spanner H of H_{net} .

Outline

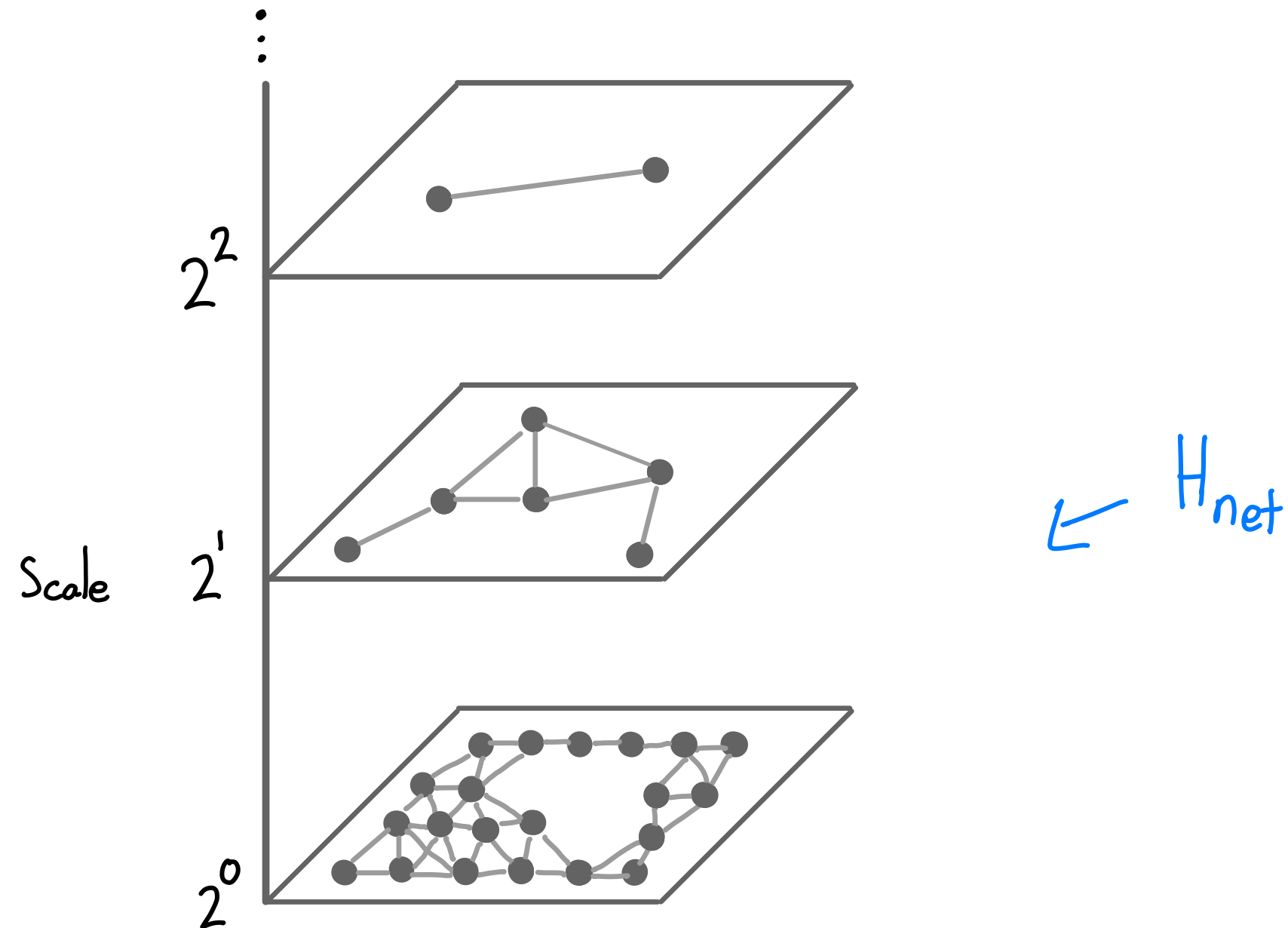
1. Background on Spanners

2. Spanner with $O(\log \Phi)$ Recourse

3. Updates in $\text{poly log } \Phi$ Time

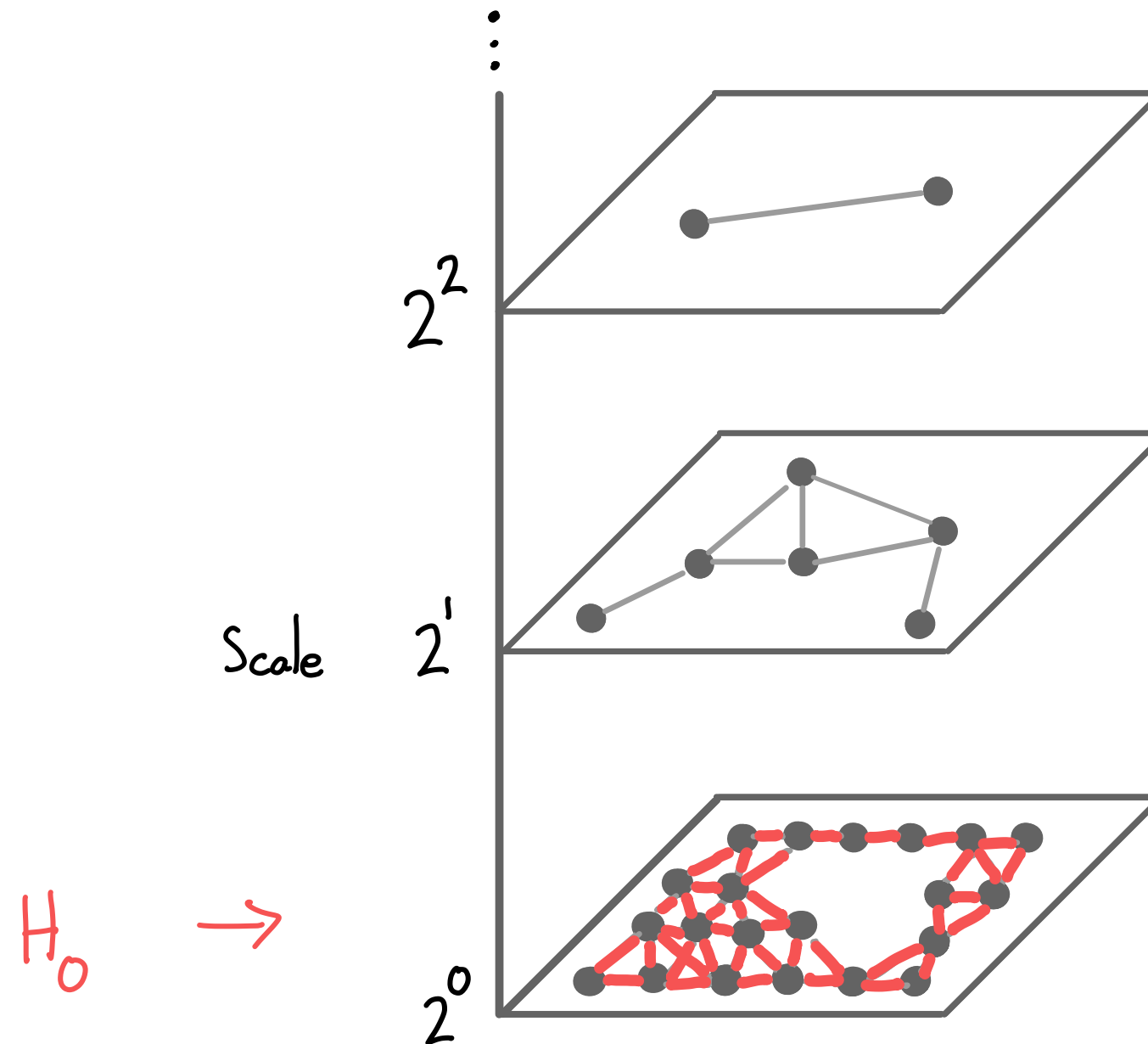
Delayed Greedy Spanner

Greedy spanner of H_{net} ...processed in batches at each scale



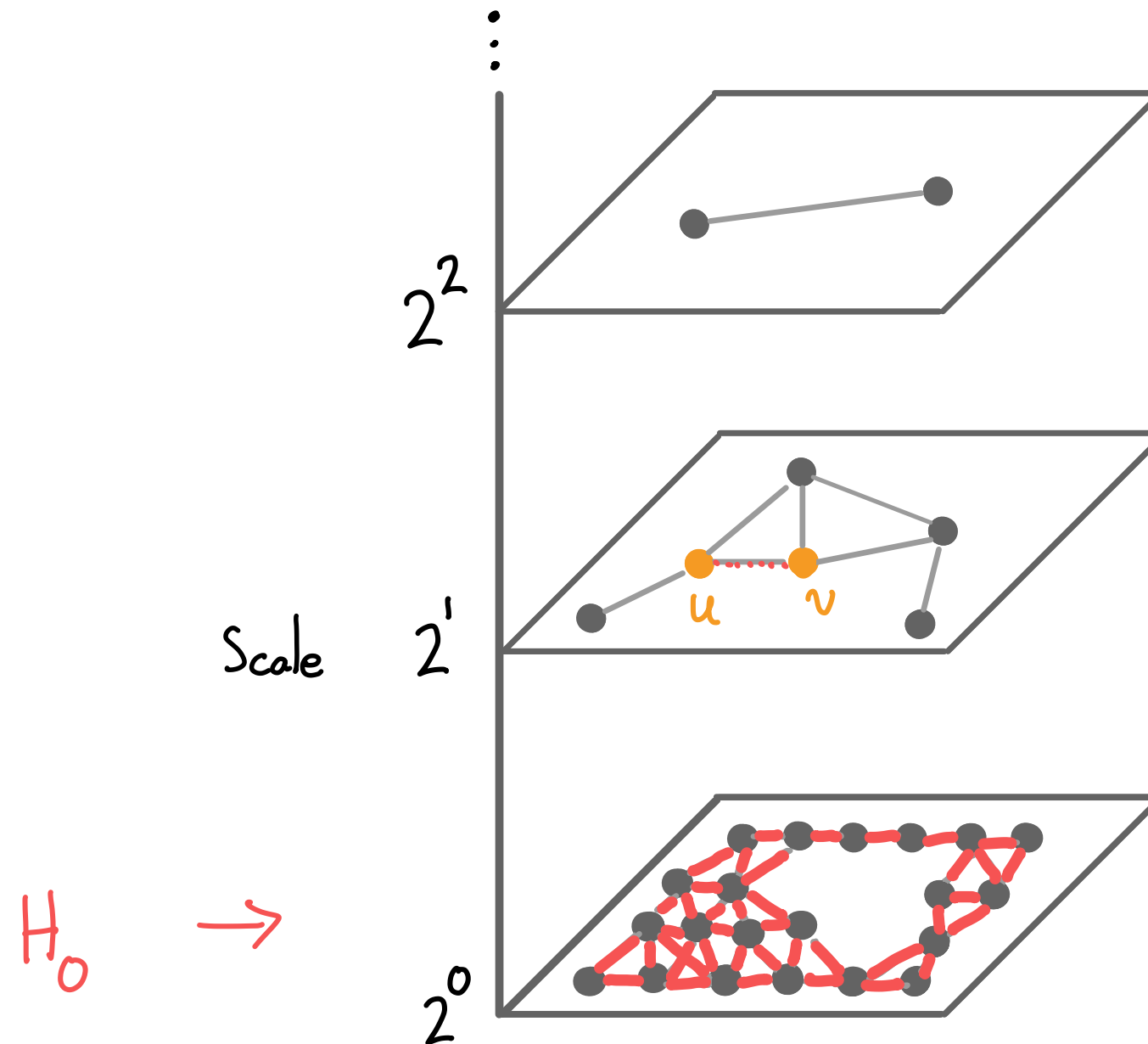
Delayed Greedy Spanner

Greedy spanner of H_{net} ...processed in batches at each scale



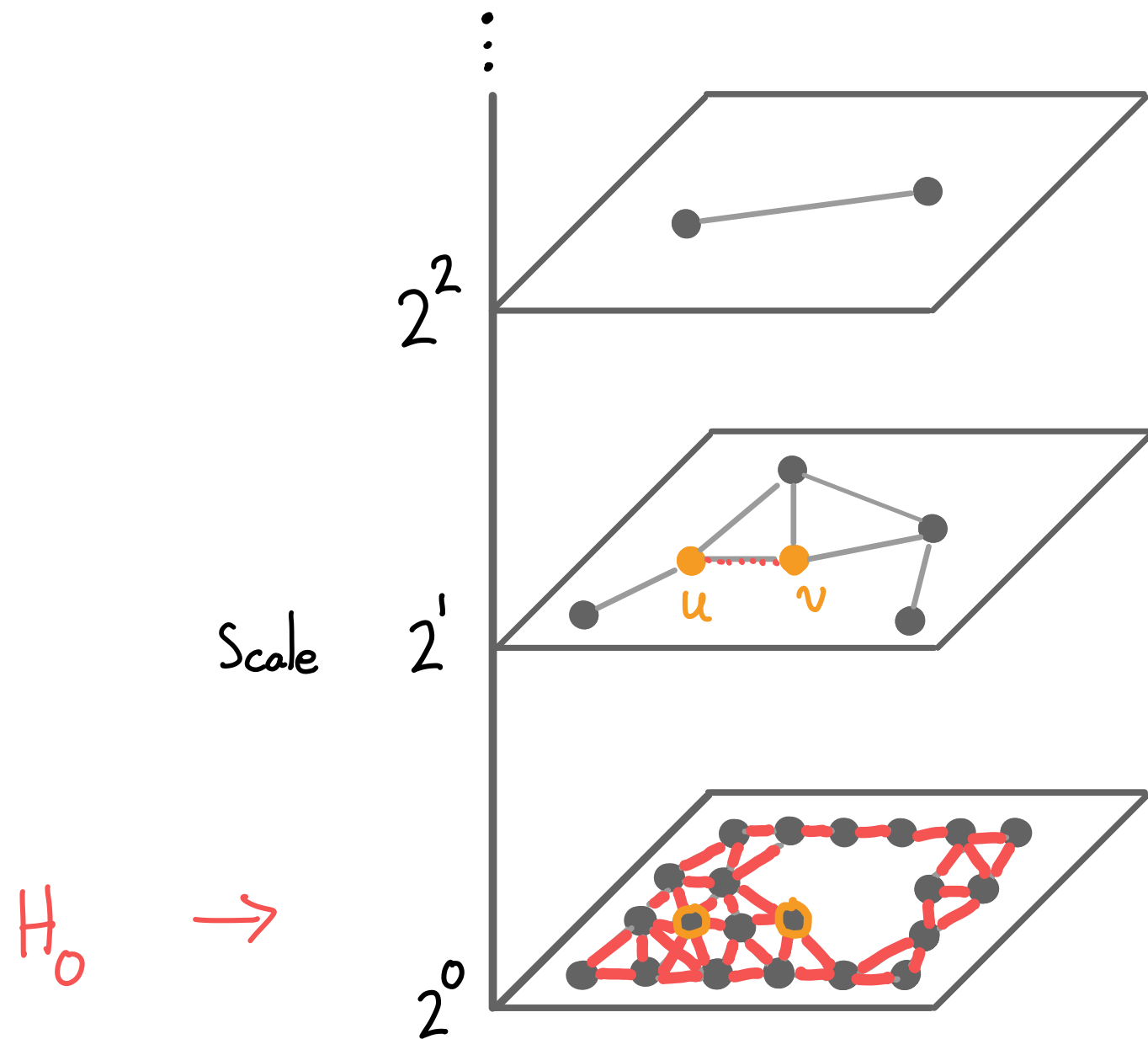
Delayed Greedy Spanner

Greedy spanner of H_{net} ...processed in batches at each scale



Delayed Greedy Spanner

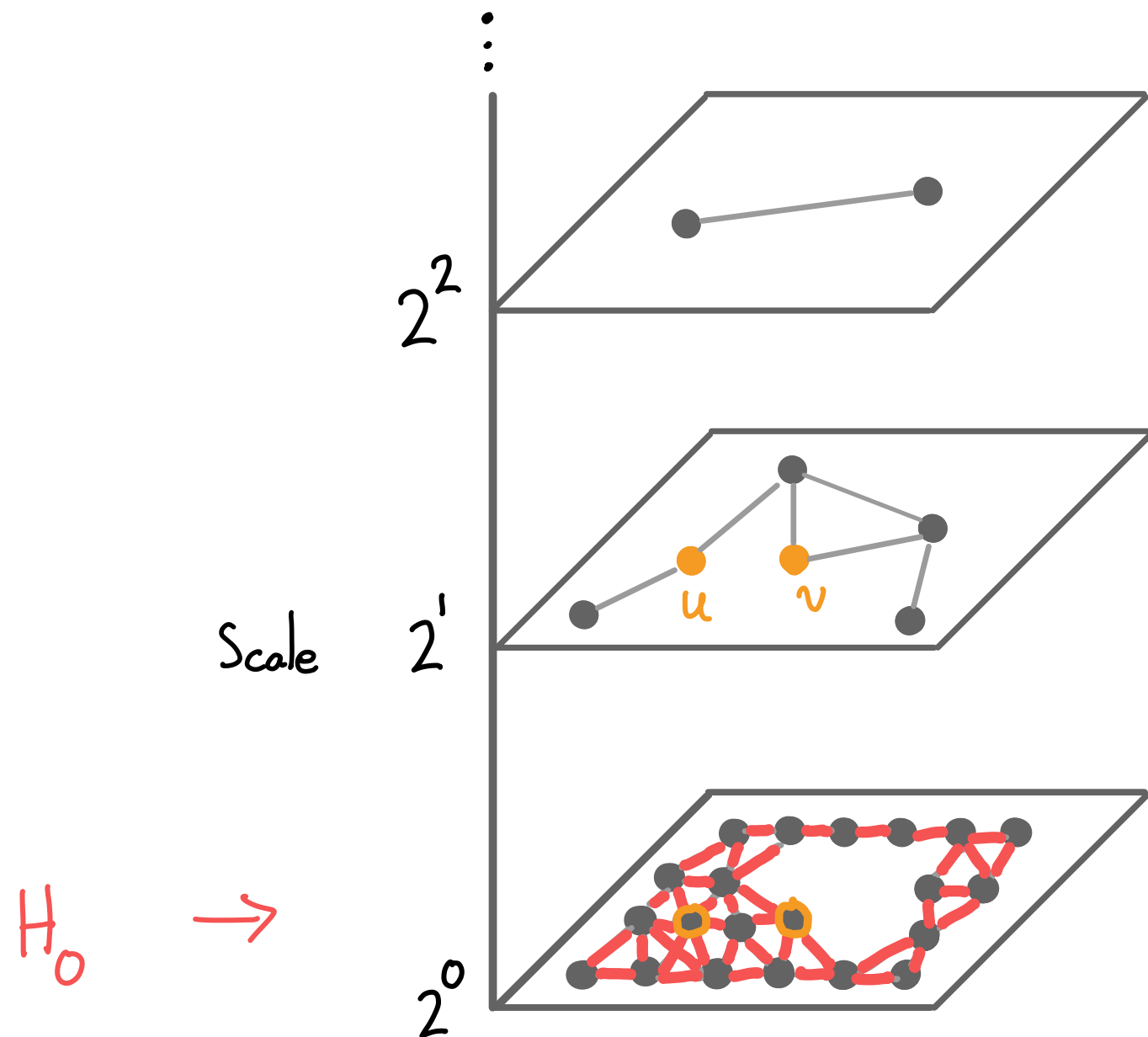
Greedy spanner of H_{net} ...processed in batches at each scale



$$d_{H_0}(u, v) > (1+\epsilon) \|u-v\| ?$$

Delayed Greedy Spanner

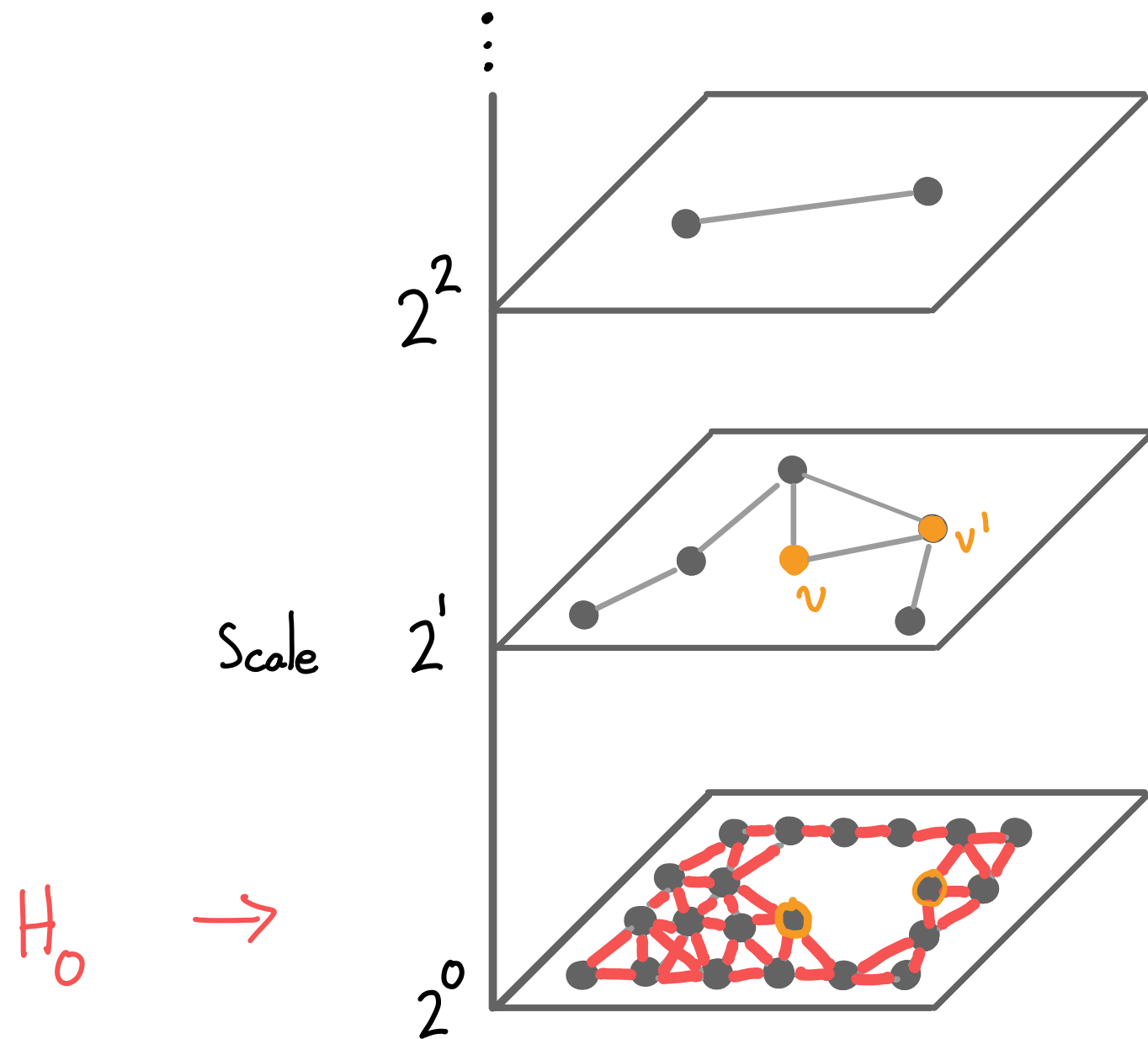
Greedy spanner of H_{net} ...processed in batches at each scale



$$d_{H_0}(u, v) > (1 + \epsilon) \|u - v\| ?$$

Delayed Greedy Spanner

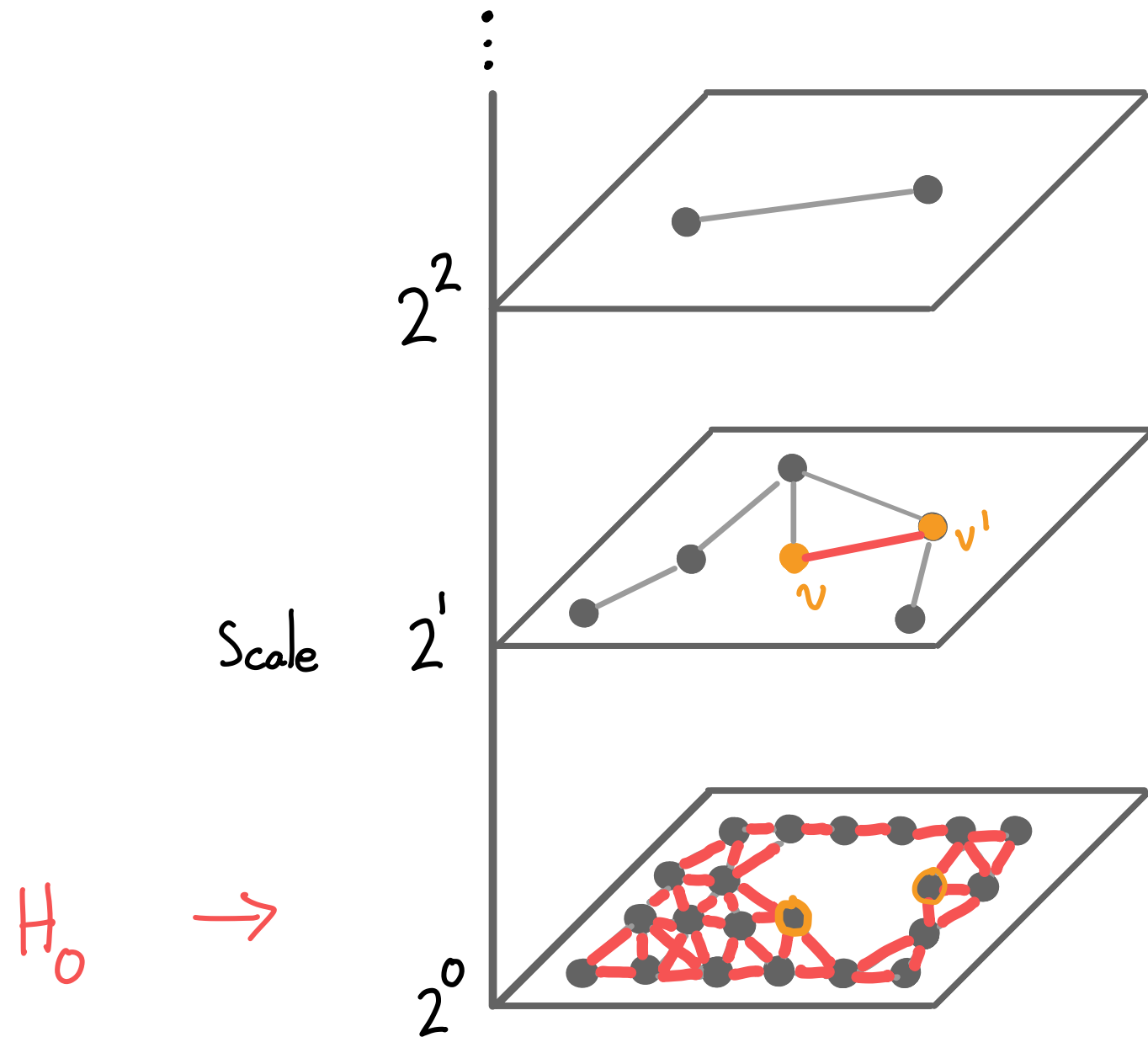
Greedy spanner of H_{net} ...processed in batches at each scale



$$d_{H_0}(v, v') > (1+\epsilon) \|v - v'\| ?$$

Delayed Greedy Spanner

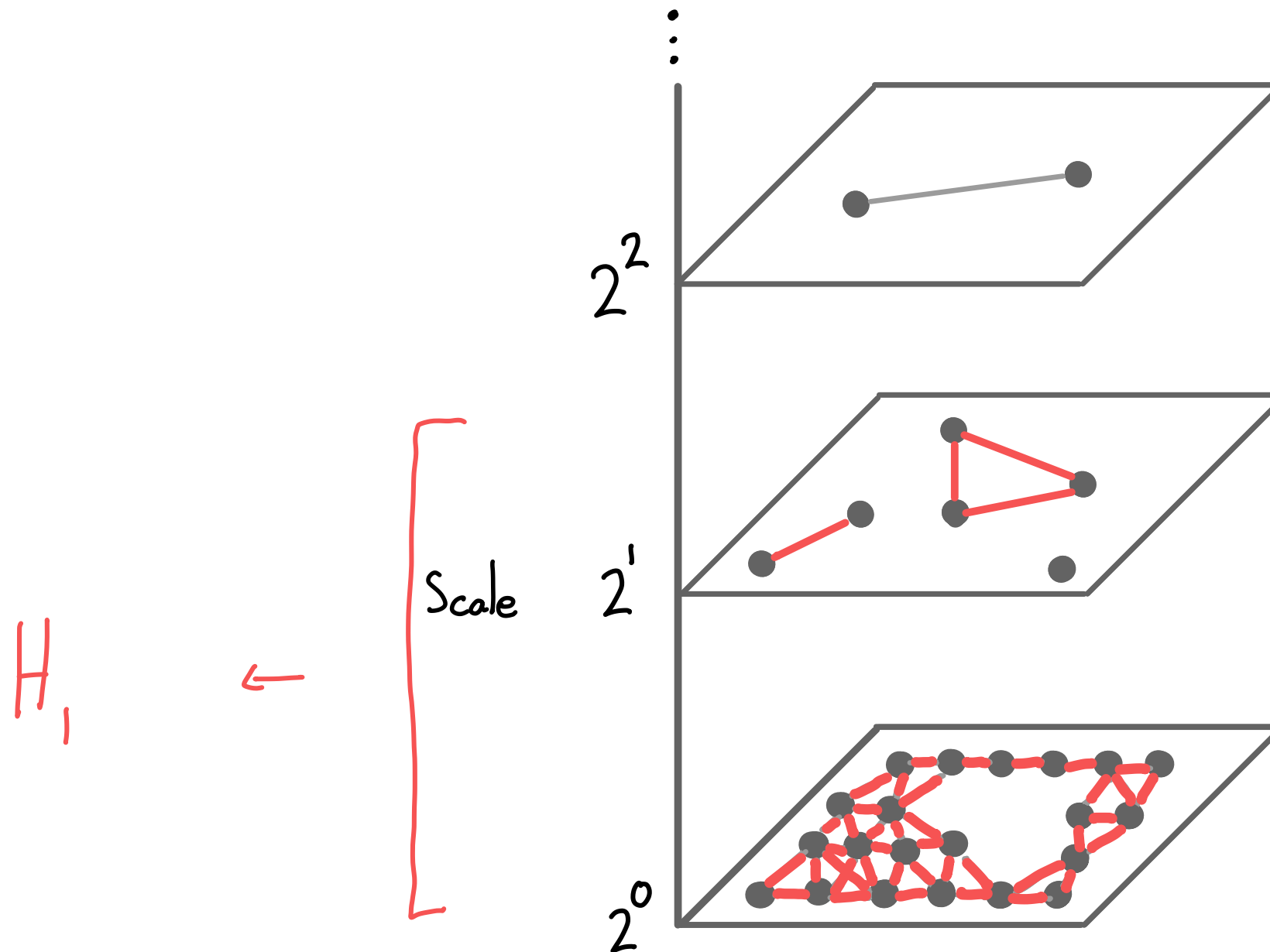
Greedy spanner of H_{net} ...processed in batches at each scale



$$d_{H_0}(v, v') > (1+\epsilon) \|v - v'\| ?$$

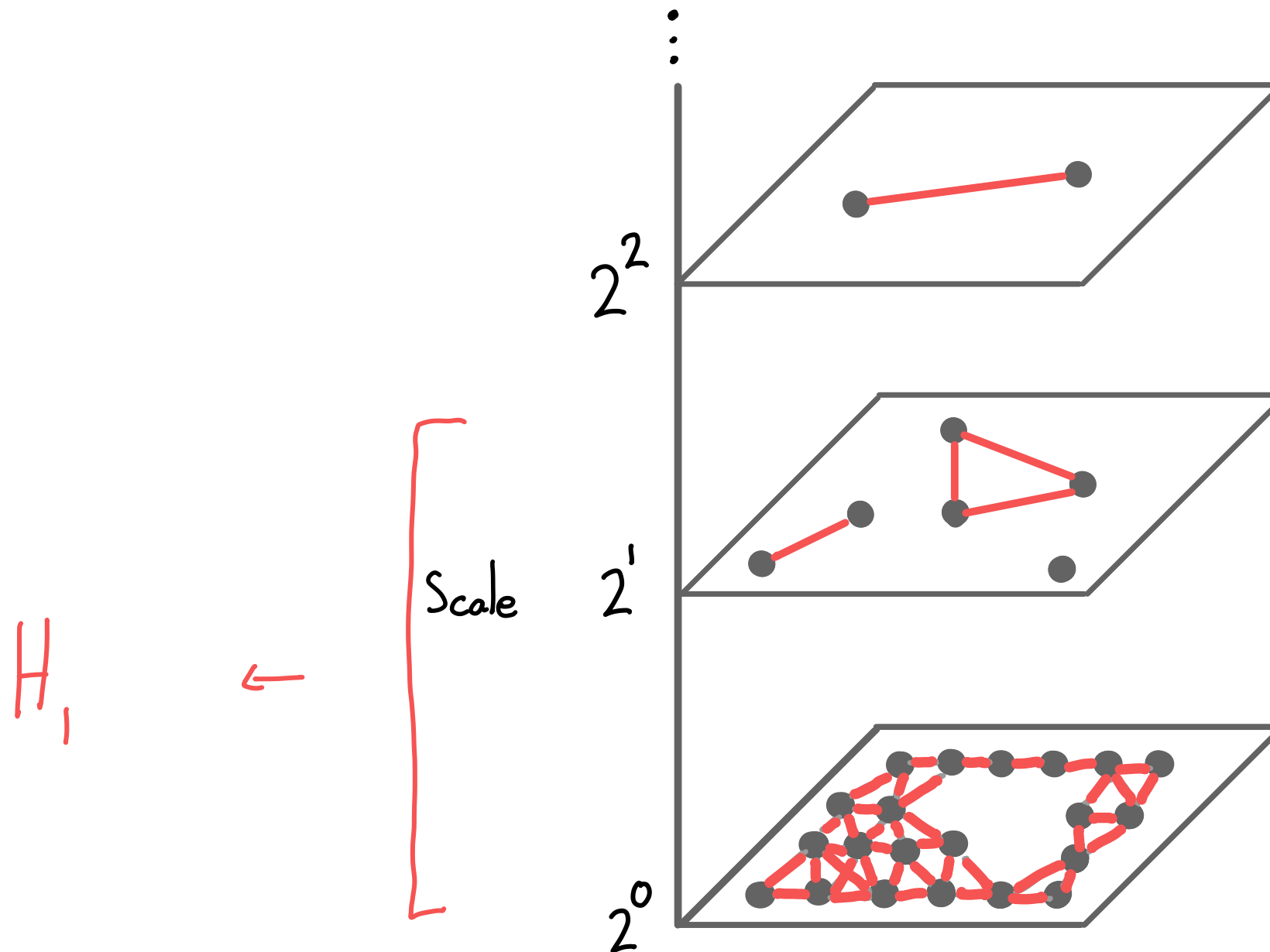
Delayed Greedy Spanner

Greedy spanner of H_{net} ...processed in batches at each scale



Delayed Greedy Spanner

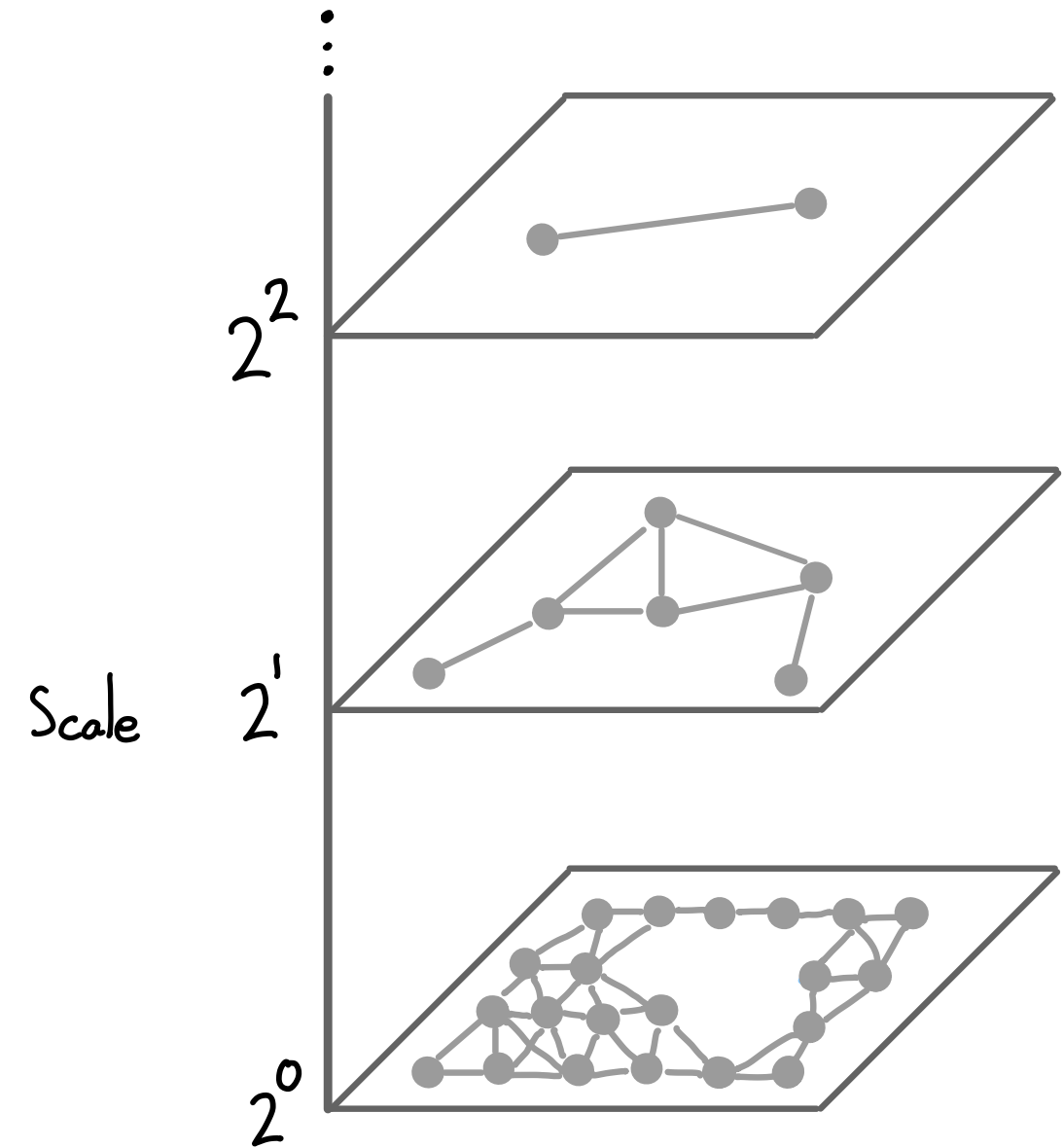
Greedy spanner of H_{net} ...processed in batches at each scale



$O(\log \Phi)$ -Recourse Insertion

1. Update H_{net}

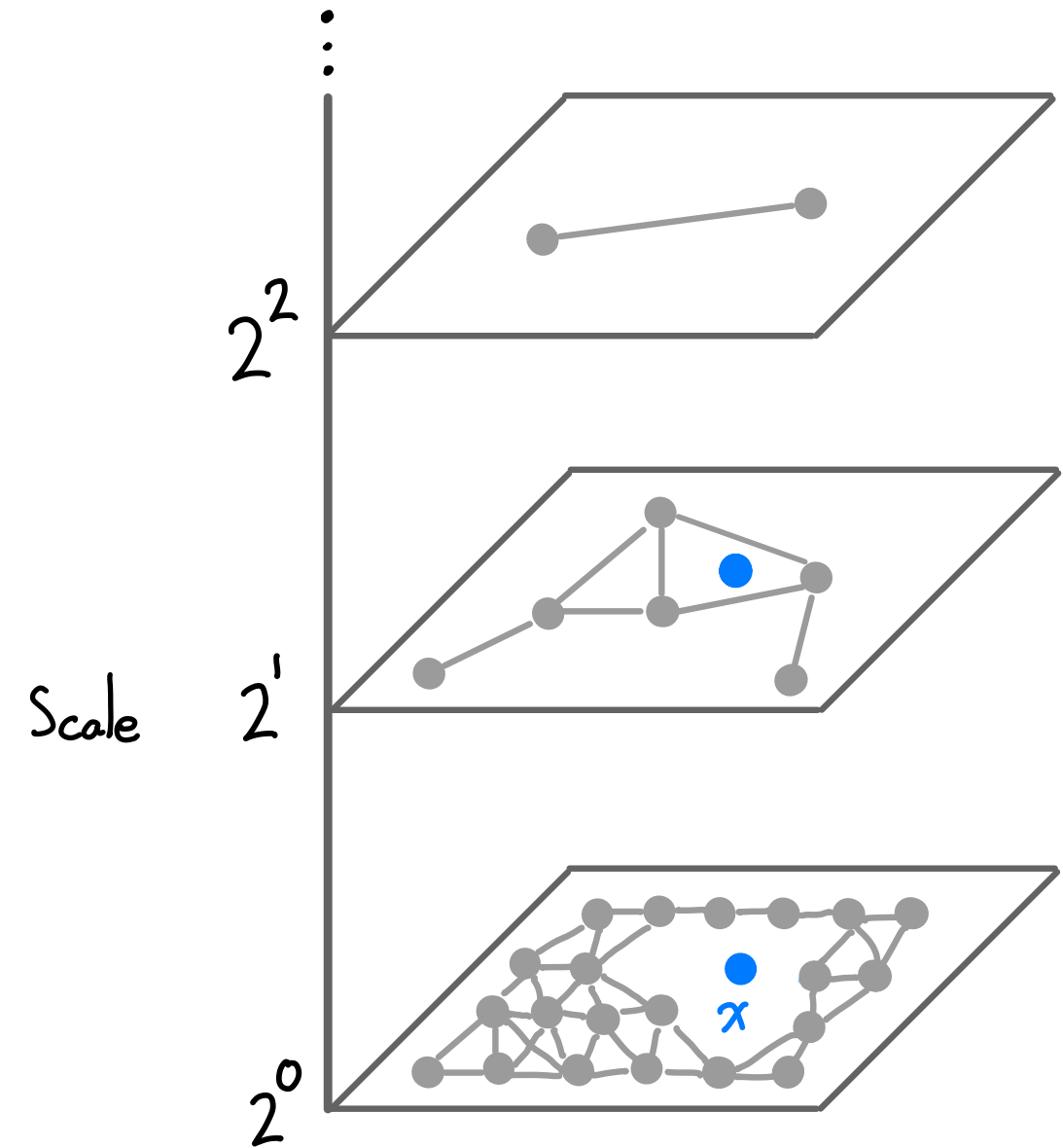
2. Update greedy spanner



$O(\log \Phi)$ -Recourse Insertion

1. Update H_{net}

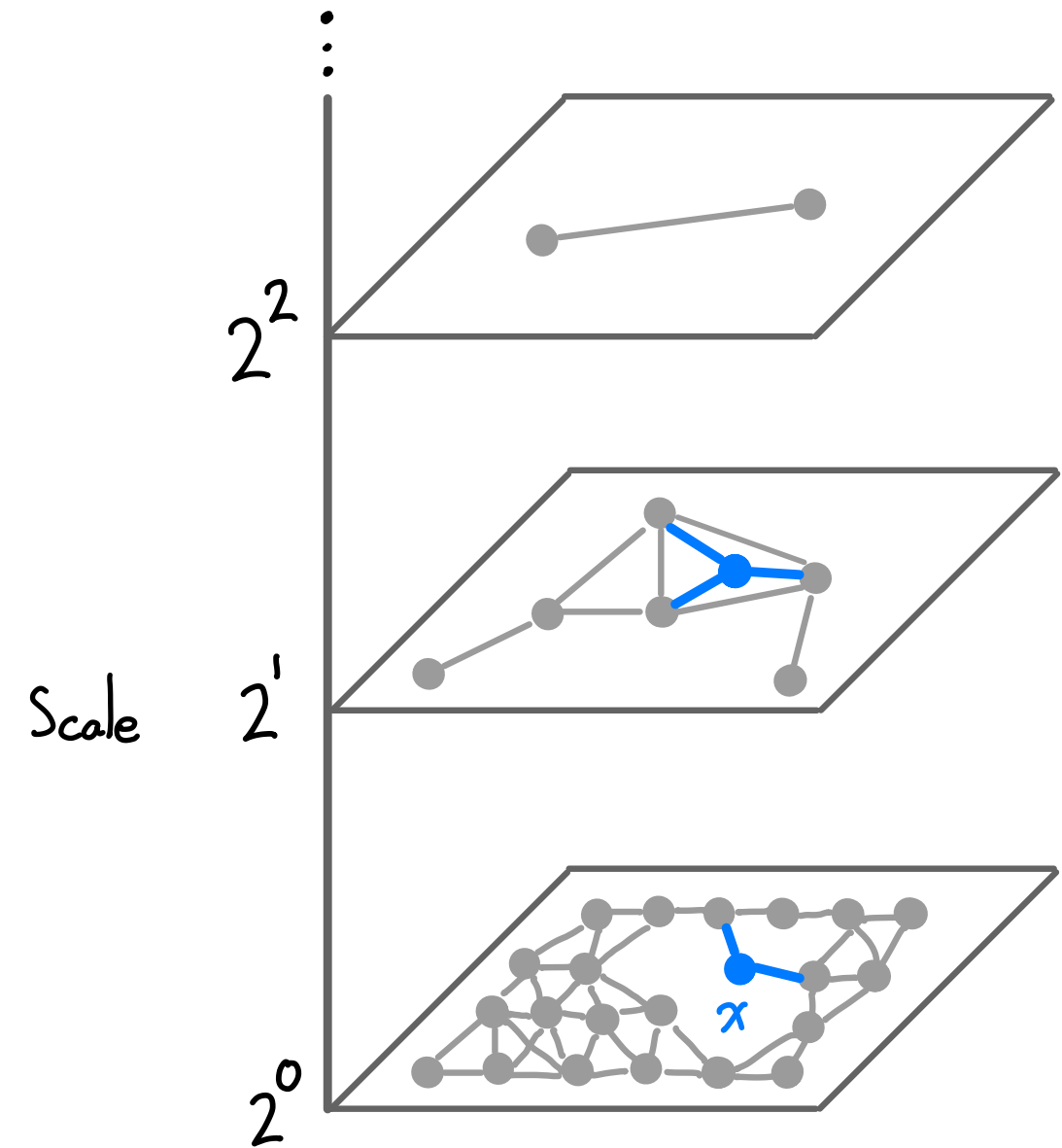
2. Update greedy spanner



$O(\log \Phi)$ -Recourse Insertion

1. Update H_{net}

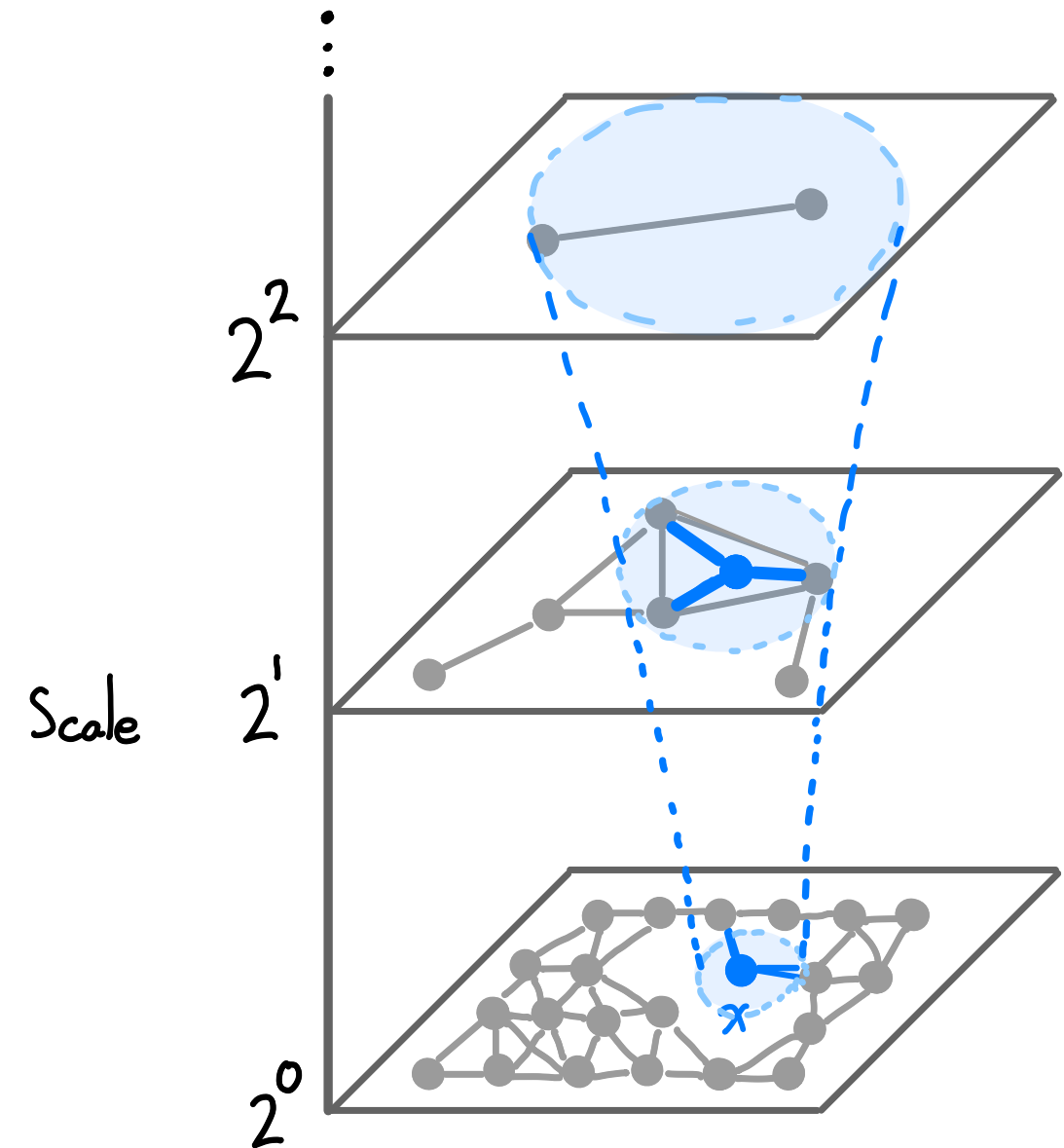
2. Update greedy spanner



$O(\log \Phi)$ -Recourse Insertion

1. Update H_{net}

2. Update greedy spanner

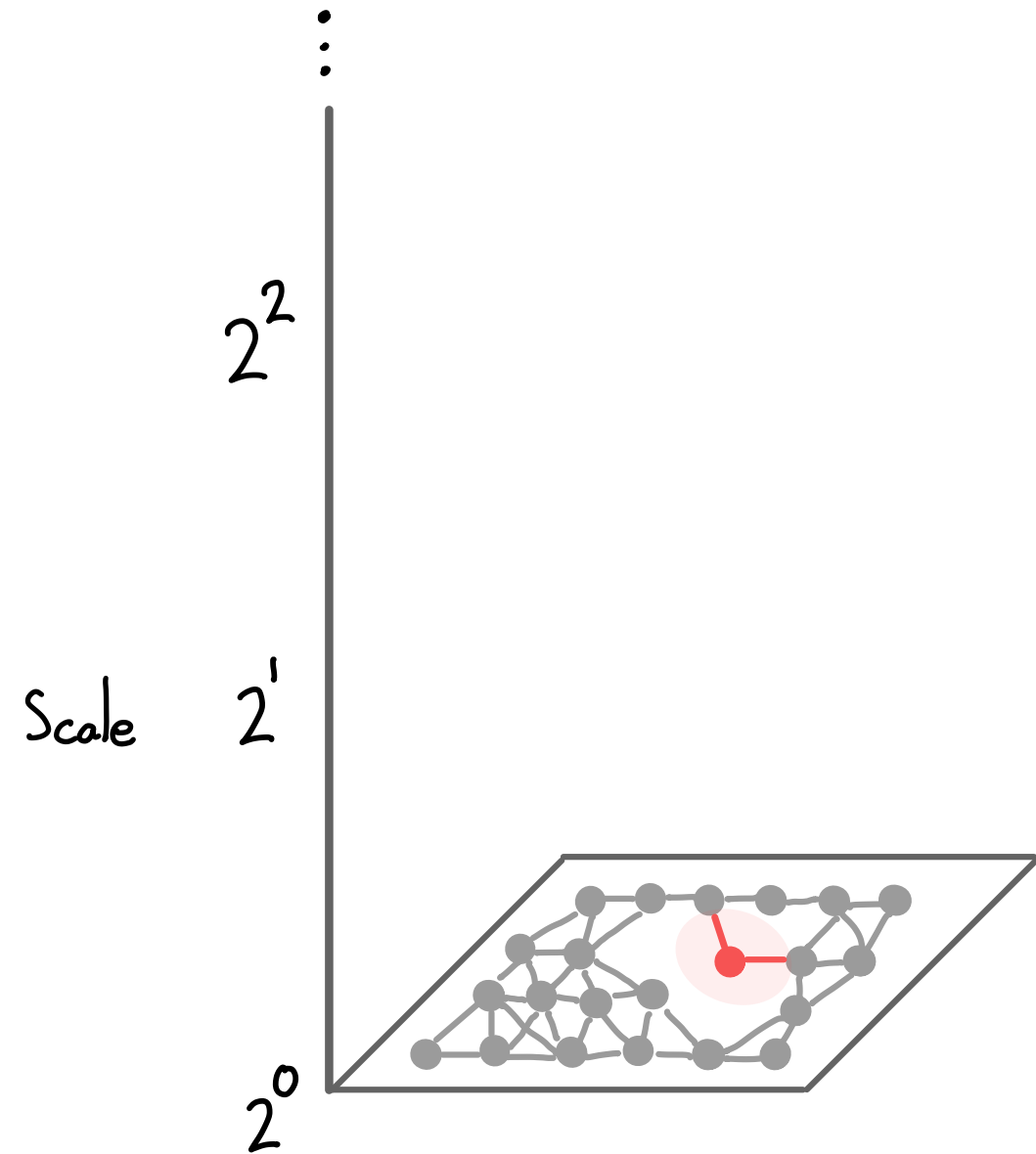


Locality: Any edge added to H_{net} is incident to x

$O(\log \Phi)$ -Recourse Insertion

1. Update H_{net}

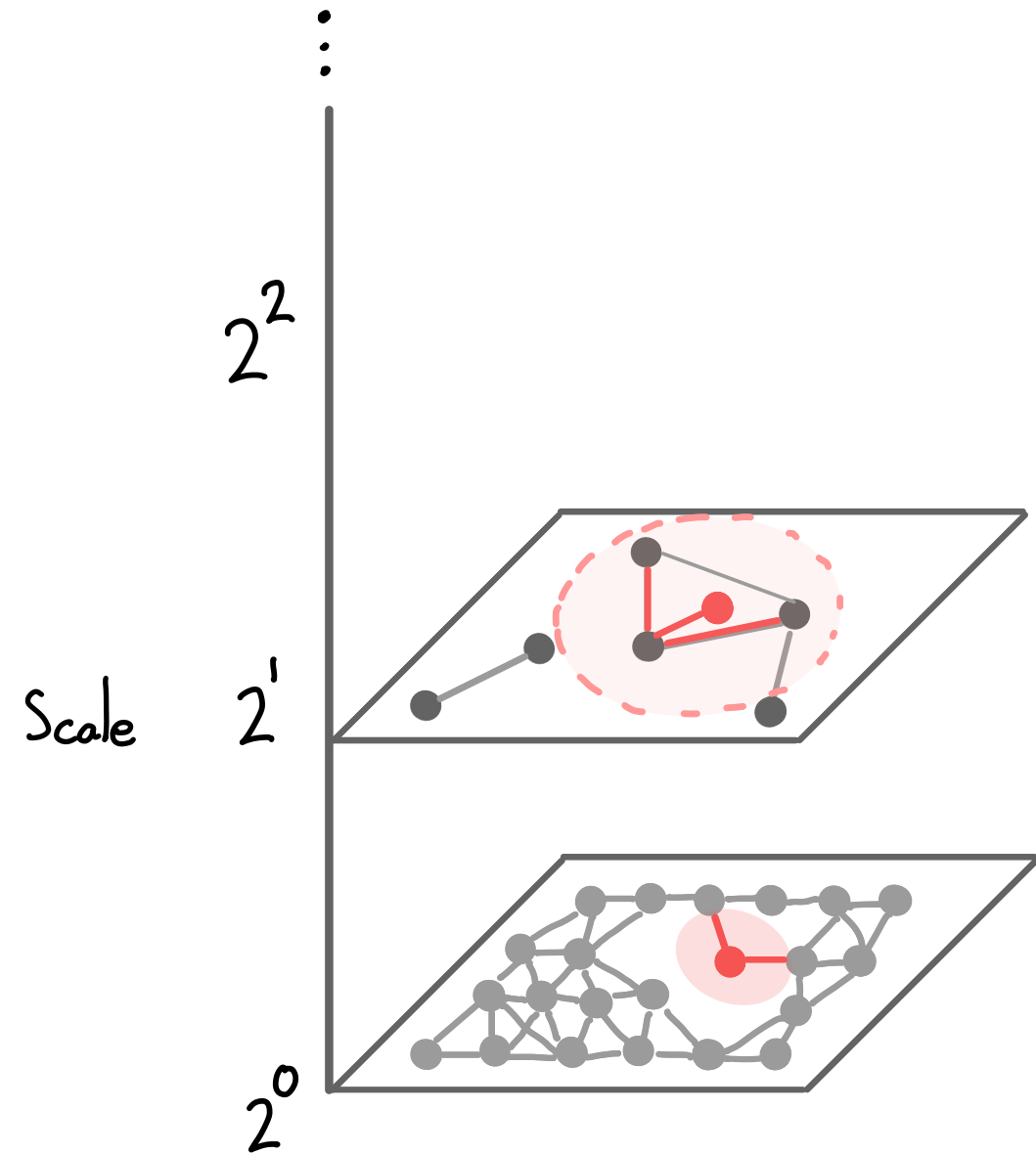
2. Update greedy spanner



$O(\log \Phi)$ -Recourse Insertion

1. Update H_{net}

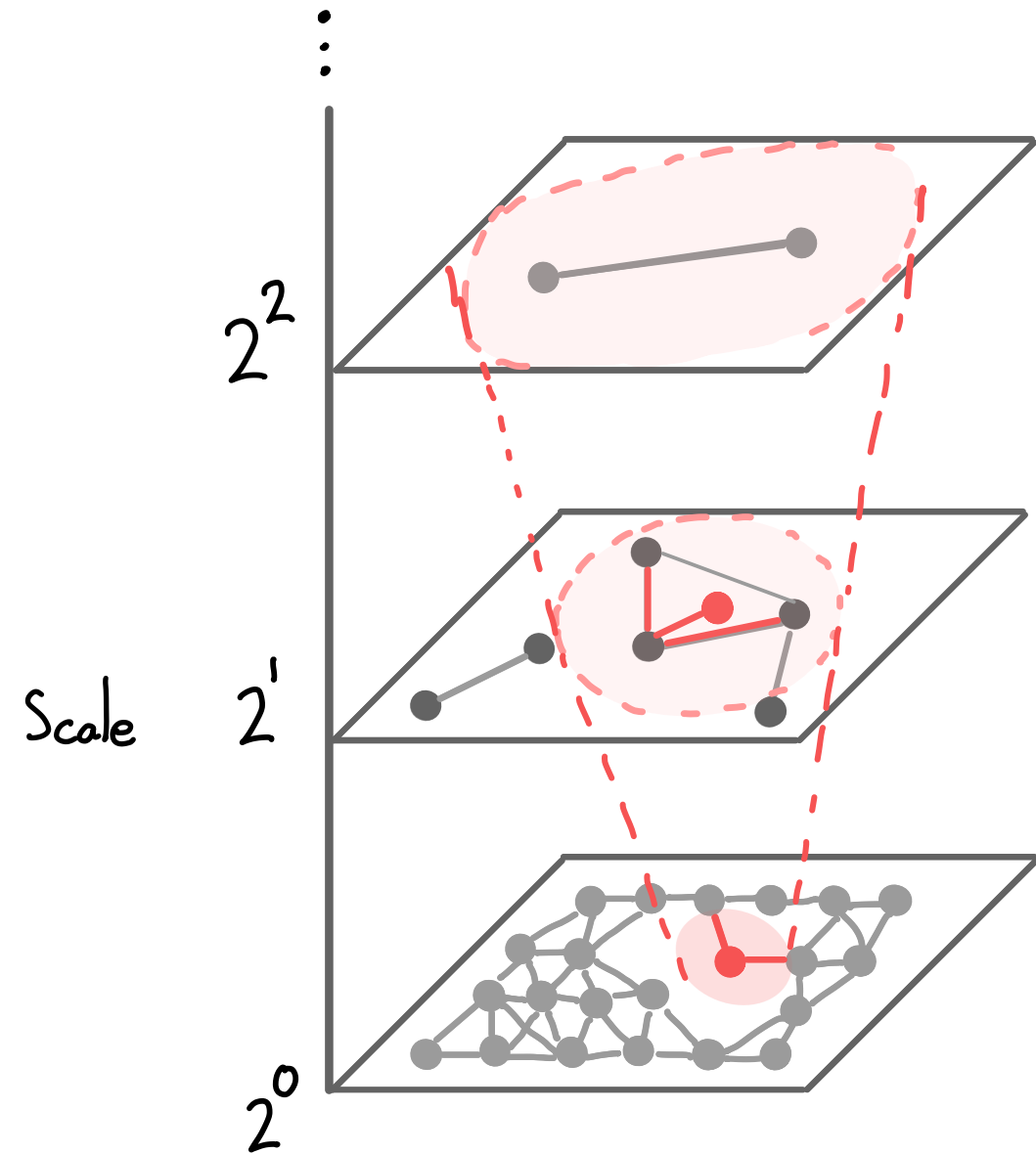
2. Update greedy spanner



$O(\log \Phi)$ -Recourse Insertion

1. Update H_{net}

2. Update greedy spanner



Key property: The only edges of H_i that change are in the ball $B(x, O(2^i))$

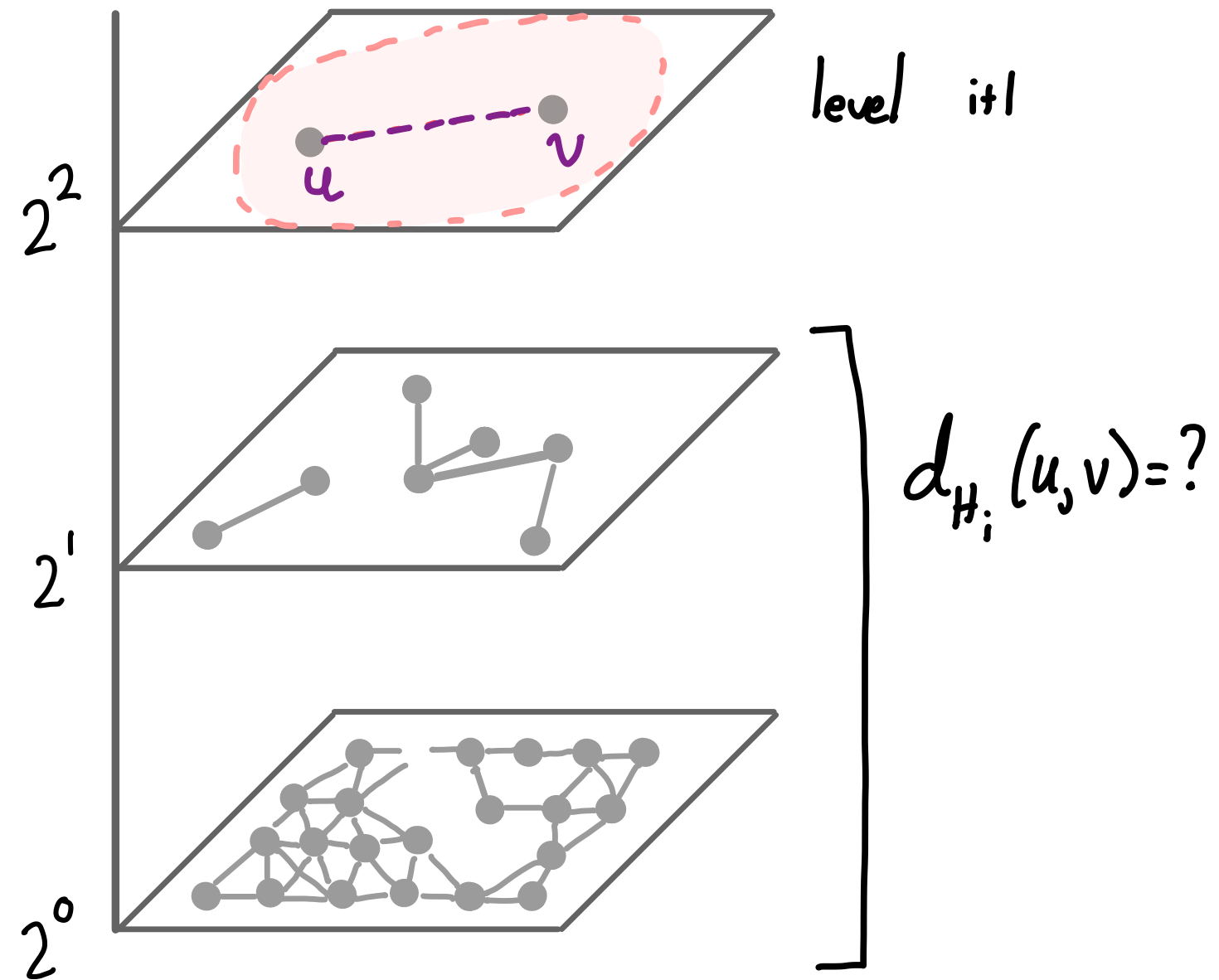
Outline

1. Background on Spanners

2. Spanner with $O(\log \Phi)$ Recourse

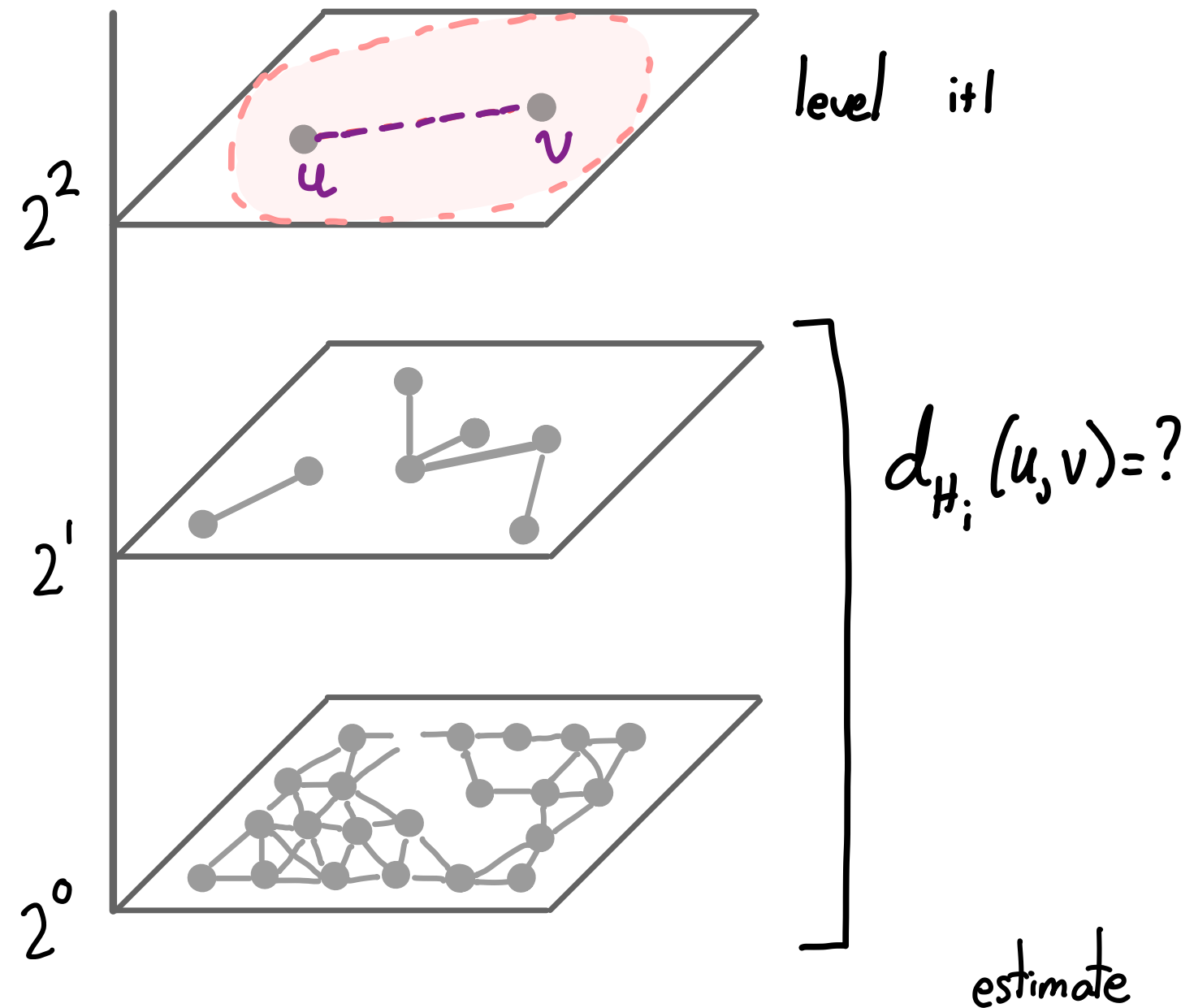
3. Updates in $\text{poly log } \Phi$ Time

Insertion in $\text{poly log } \Phi$ Time



Bottleneck: for $O_d(1)$ edges at scale $i + 1$, recompute $d_{H_i}(u, v)$

Insertion in $\text{poly log } \Phi$ Time

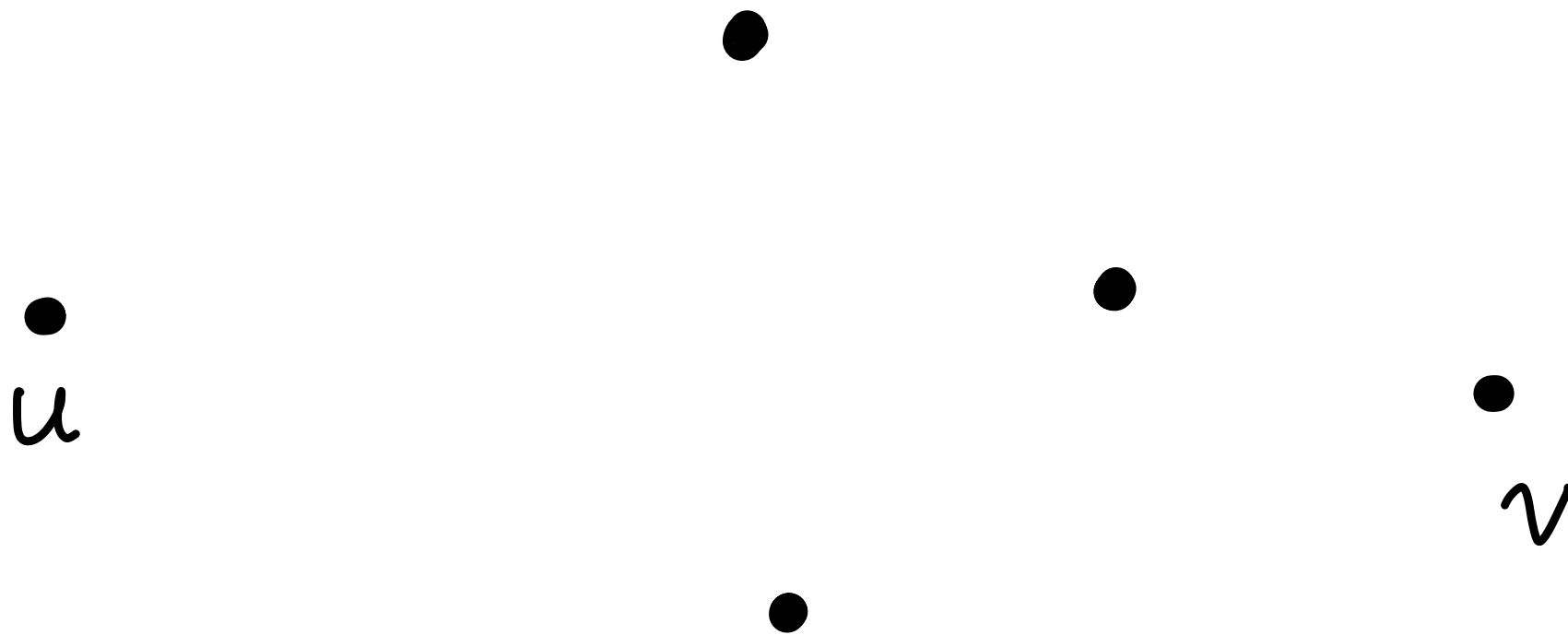


Bottleneck: for $O_d(1)$ edges at scale $i + 1$, ~~recompute~~ $d_{H_i}(u, v)$

Insertion in $\text{poly log } \Phi$ Time

We compute $d_{H_i}(u, v)$ using:

1. Inductively maintained estimates of $d_{H_{i-1}}(a, b)$ for any pair (a, b)
2. $O_d(1)$ relevant edges at scale i

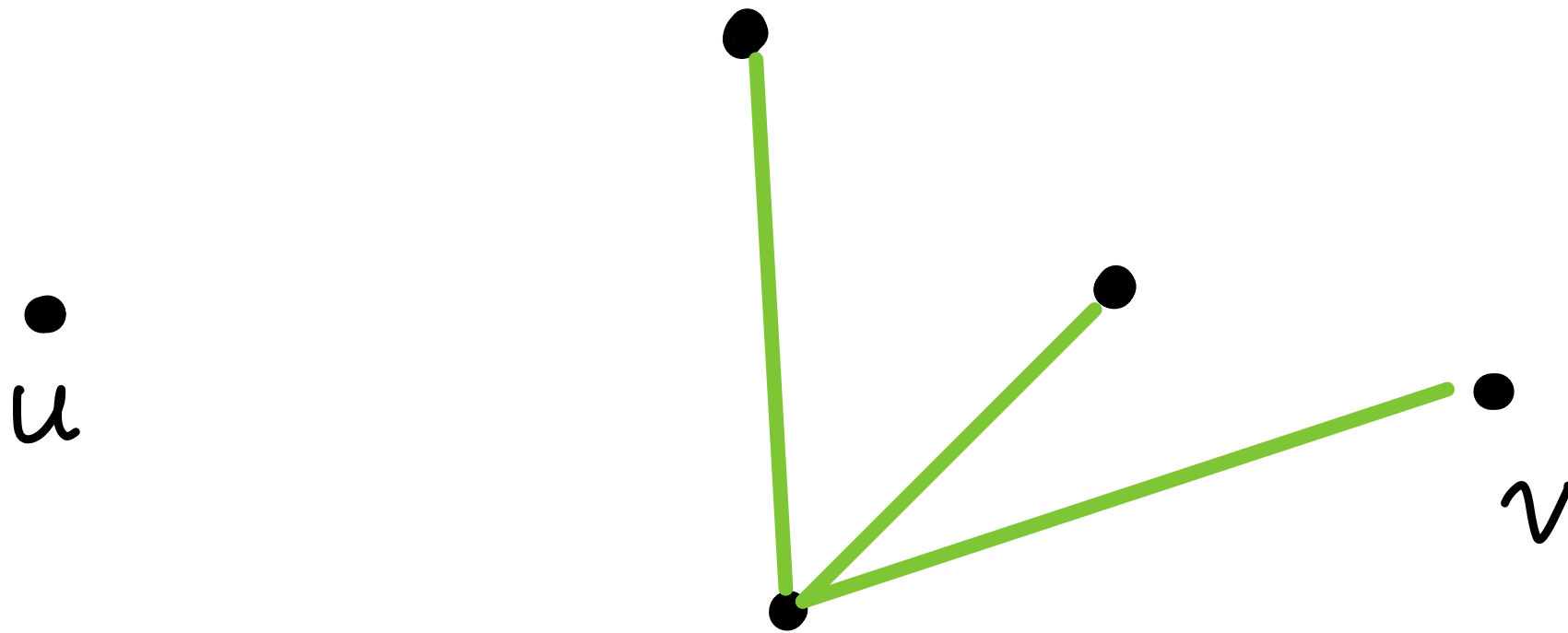


$$H_i = (H_i \setminus H_{i-1}) \cup H_{i-1}$$

Insertion in $\text{poly log } \Phi$ Time

We compute $d_{H_i}(u, v)$ using:

1. Inductively maintained estimates of $d_{H_{i-1}}(a, b)$ for any pair (a, b)
2. $O_d(1)$ relevant edges at scale i

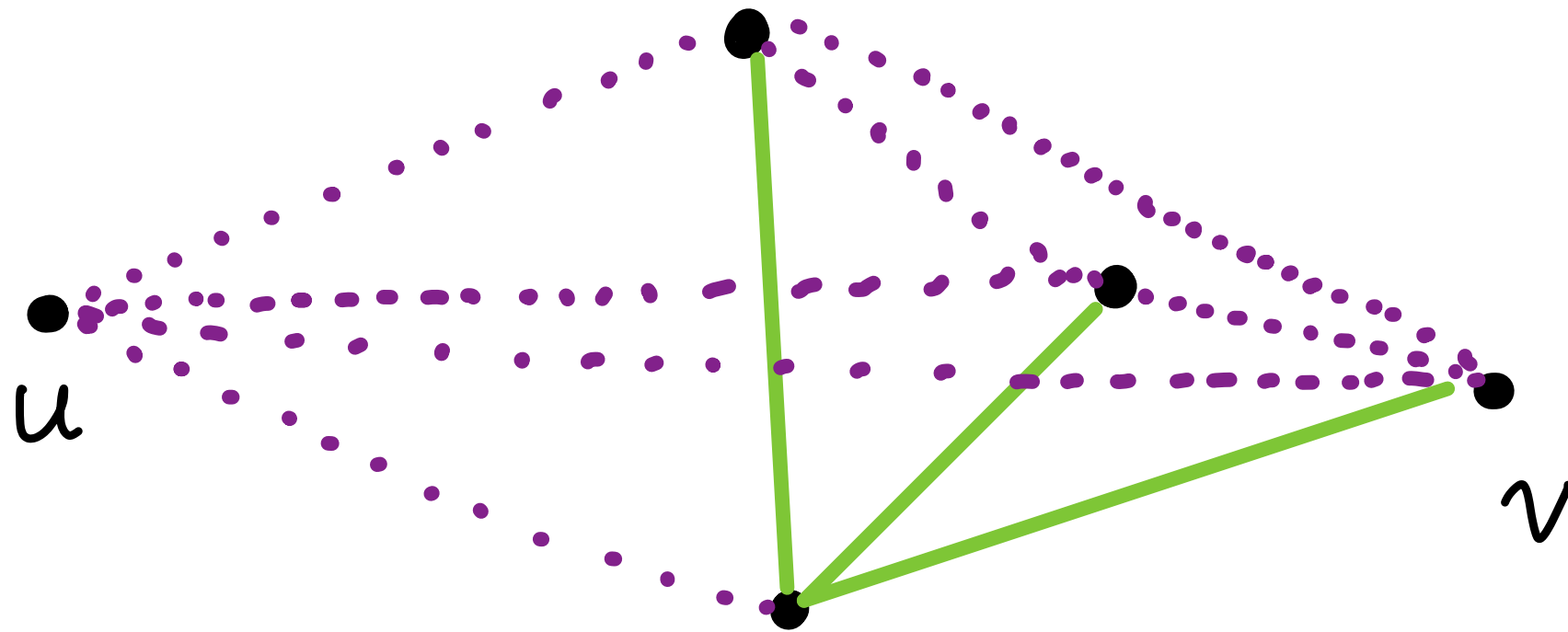


$$H_i = (H_i \setminus H_{i-1}) \cup H_{i-1}$$

Insertion in $\text{poly log } \Phi$ Time

We compute $d_{H_i}(u, v)$ using:

1. Inductively maintained estimates of $d_{H_{i-1}}(a, b)$ for any pair (a, b)
2. $O_d(1)$ relevant edges at scale i

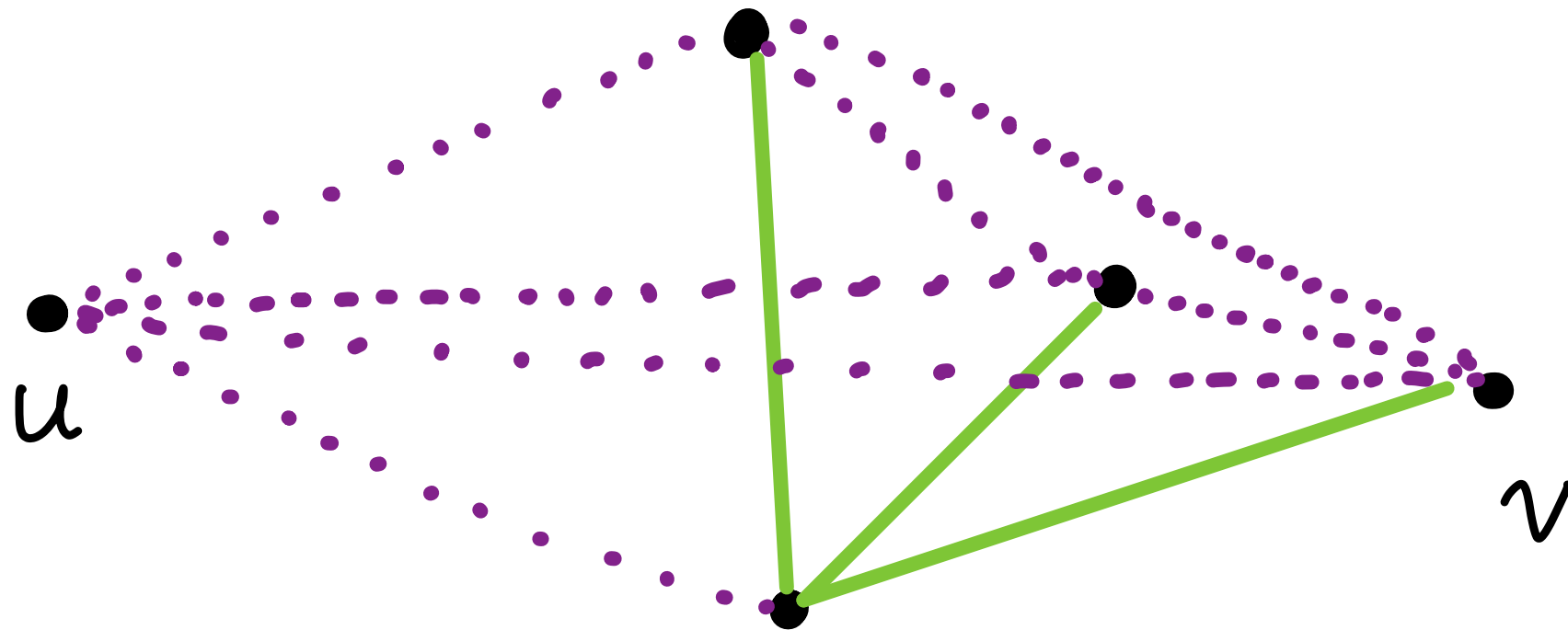


$$H_i = (H_i \setminus H_{i-1}) \cup H_{i-1}$$

Insertion in $\text{poly } \log \Phi$ Time

We compute $d_{H_i}(u, v)$ using:

1. Inductively maintained estimates of $d_{H_{i-1}}(a, b)$ for any pair (a, b)
2. $O_d(1)$ relevant edges at scale i

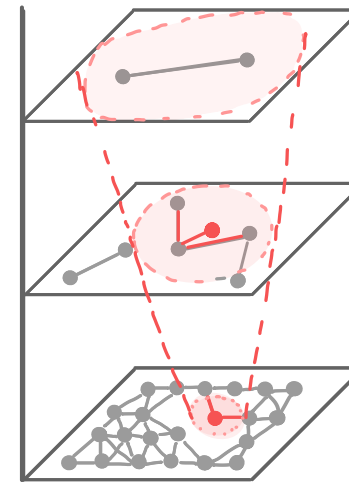
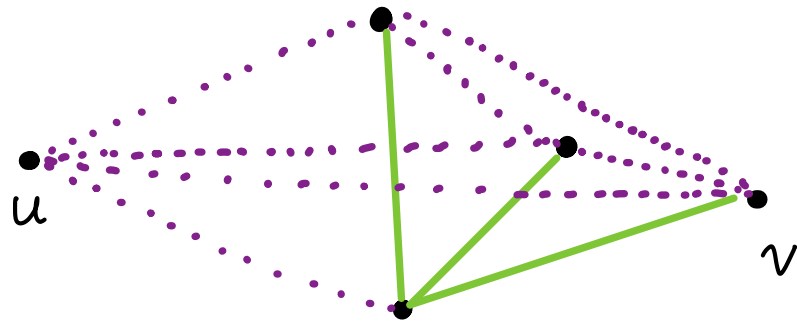


Caution: Error accumulates at $\log \Phi$ scales, so need $(1 + \frac{\varepsilon}{\log \Phi})$ -approximation.
So, $(\log \Phi)^{O(d)}$ runtime.

Conclusion

Summary:

- **Result:** A light $(1 + \varepsilon)$ -spanner for dynamic points in $\mathbb{R}^d$ in $\text{poly log } \Phi$ time.
- **Idea:** Delayed Greedy Spanner only changes in a “cone” around updated point x . Update quickly using inductive estimates.



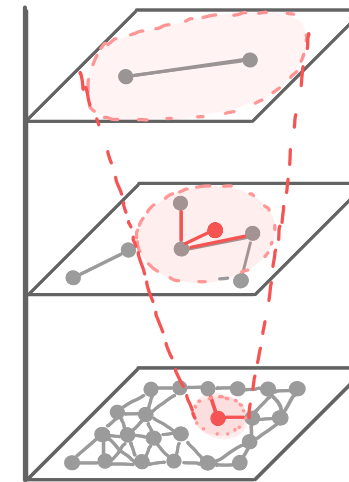
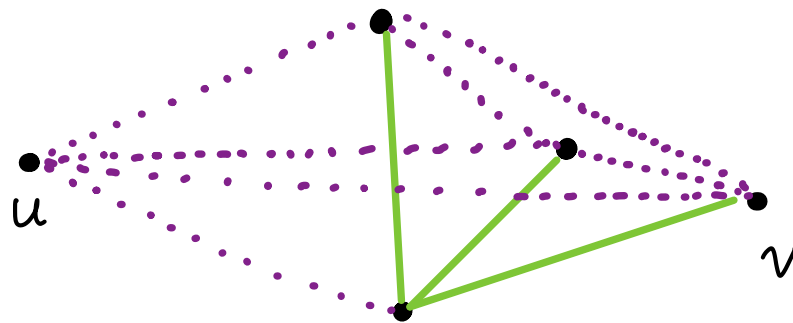
Open problems:

- Remove the dependence on Φ , and get $\text{poly log } n$?
- Reduce update time to $O(\log \Phi)$ instead of $(\log \Phi)^{O(d)}$?

Conclusion

Summary:

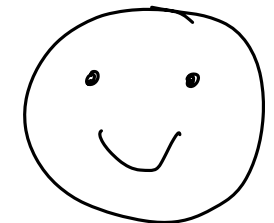
- **Result:** A light $(1 + \varepsilon)$ -spanner for dynamic points in $\mathbb{R}^d$ in $\text{poly log } \Phi$ time.
- **Idea:** Delayed Greedy Spanner only changes in a “cone” around updated point x . Update quickly using inductive estimates.



Thank you!

Open problems:

- Remove the dependence on Φ , and get $\text{poly log } n$?
- Reduce update time to $O(\log \Phi)$ instead of $(\log \Phi)^{O(d)}$?



I'm on the job market!